

```
0001 import { _decorator, Camera, Component, Vec3, view } from 'cc';
0002 import { boundsForCells } from './shared/game/index';
0003 import type { LevelDefinition } from './shared/game/index';
0004
0005 const { ccclass, property } = _decorator;
0006
0007 @ccclass('CameraController')
0008 export class CameraController extends Component {
0009   @property(Camera)
0010   camera: Camera | null = null;
0011
0012   frameLevel(level: LevelDefinition, horizontalScreenOffset = 0): void {
0013     const cells = [
0014       ...level.tiles,
0015       ...((level.bridges ?? []).reduce<Array<{ x: number; z: number }>>(
0016         (result, bridge) => result.concat(bridge.cells),
0017         [],
0018       )),
0019       ...((level.splits ?? []).reduce<Array<{ x: number; z: number }>>(
0020         (result, split) => result.concat(split.destinations),
0021         [],
0022       )),
0023       level.goal,
0024     ];
0025     const bounds = boundsForCells(cells);
0026     const centerX = (bounds.minX + bounds.maxX) * 0.5;
0027     const centerZ = (bounds.minZ + bounds.maxZ) * 0.5;
0028     const target = new Vec3(centerX, 0, centerZ);
0029
0030     // This fixed offset gives approximately 34 degrees pitch and 18 degrees yaw.
0031     const cameraOffset = new Vec3(-6, 13, -18);
0032     const basePosition = target.clone().add(cameraOffset);
0033     const orthoHeight = this.orthoHeightForBounds(
0034       bounds,
0035       target,
0036       basePosition,
0037       horizontalScreenOffset,
0038     );
0039     const visible = view.getVisibleSize();
0040     const aspect = Math.max(1, visible.width / visible.height);
0041     const forward = target.clone().subtract(basePosition).normalize();
0042     const right = new Vec3();
0043     Vec3.cross(right, forward, Vec3.UP).normalize();
0044     const framedTarget = new Vec3();
0045     Vec3.scaleAndAdd(
0046       framedTarget,
0047       target,
0048       right,
0049       -orthoHeight * aspect * horizontalScreenOffset,
0050     );
0051     const position = framedTarget.clone().add(cameraOffset);
0052     this.node.setPosition(position);
```

```
0053     this.node.lookAt(framedTarget, Vec3.UP);
0054
0055     if (!this.camera) {
0056         return;
0057     }
0058     this.camera.projection = Camera.ProjectionType.ORTHO;
0059     this.camera.orthoHeight = orthoHeight;
0060 }
0061
0062 private orthoHeightForBounds(
0063     bounds: ReturnType<typeof boundsForCells>,
0064     target: Vec3,
0065     cameraPosition: Vec3,
0066     horizontalScreenOffset: number,
0067 ): number {
0068     const forward = target.clone().subtract(cameraPosition).normalize();
0069     const right = new Vec3();
0070     Vec3.cross(right, forward, Vec3.UP);
0071     right.normalize();
0072     const cameraUp = new Vec3();
0073     Vec3.cross(cameraUp, right, forward);
0074     cameraUp.normalize();
0075
0076     let horizontalExtent = 0;
0077     let verticalExtent = 0;
0078     const xValues = [bounds.minX - 0.55, bounds.maxX + 0.55];
0079     const yValues = [-0.75, 2.15];
0080     const zValues = [bounds.minZ - 0.55, bounds.maxZ + 0.55];
0081     for (const x of xValues) {
0082         for (const y of yValues) {
0083             for (const z of zValues) {
0084                 const relative = new Vec3(x, y, z).subtract(target);
0085                 horizontalExtent = Math.max(horizontalExtent, Math.abs(Vec3.dot(relative, right)));
0086                 verticalExtent = Math.max(verticalExtent, Math.abs(Vec3.dot(relative, cameraUp)));
0087             }
0088         }
0089     }
0090
0091     const visible = view.getVisibleSize();
0092     const aspect = Math.max(1, visible.width / visible.height);
0093     const verticalWithMargin = verticalExtent + 0.9;
0094     const visibleHorizontalRatio = Math.max(0.35, 1 - Math.abs(horizontalScreenOffset));
0095     const horizontalWithMargin = horizontalExtent / (aspect * visibleHorizontalRatio) + 0.9;
0096     return Math.max(3.4, verticalWithMargin, horizontalWithMargin);
0097 }
0098 }
0099
0100 import { _decorator, AudioClip, AudioSource, Component, Node, resources } from 'cc';
0101
0102 const { ccclass } = _decorator;
0103
0104 type EffectName = 'move1' | 'move2' | 'move3' | 'fall' | 'glass' | 'complete' | 'ui';
```

```
0007 @ccclass('AudioController')
0008 export class AudioController extends Component {
0009   private effectsSource: AudioSource | null = null;
0010   private ambientSource: AudioSource | null = null;
0011   private readonly effects = new Map<EffectName, AudioClip>();
0012   private moveIndex = 0;
0013   private ambientRequested = false;
0014   private soundEnabled = true;
0015
0016   start(): void {
0017     this.effectsSource = this.createSource('EffectsAudio');
0018     this.ambientSource = this.createSource('AmbientAudio');
0019     this.ambientSource.loop = true;
0020     this.ambientSource.volume = 0.32;
0021
0022     this.loadEffect('audio/move-stone-1', 'move1');
0023     this.loadEffect('audio/move-stone-2', 'move2');
0024     this.loadEffect('audio/move-stone-3', 'move3');
0025     this.loadEffect('audio/fall', 'fall');
0026     this.loadEffect('audio/glass-break', 'glass');
0027     this.loadEffect('audio/complete', 'complete');
0028     this.loadEffect('audio/ui-click', 'ui');
0029     resources.load('audio/ambient-loop', AudioClip, (error, clip) => {
0030       if (error || !this.ambientSource) {
0031         console.warn('Ambient audio failed to load.', error);
0032         return;
0033       }
0034       this.ambientSource.clip = clip;
0035       if (this.ambientRequested) {
0036         this.ambientSource.play();
0037       }
0038     });
0039   }
0040
0041   playMove(): void {
0042     const names: readonly EffectName[] = ['move1', 'move2', 'move3'];
0043     this.play(names[this.moveIndex % names.length], 0.68);
0044     this.moveIndex += 1;
0045   }
0046
0047   beginInteraction(): void {
0048     this.ensureAmbient();
0049   }
0050
0051   playFall(): void {
0052     this.play('fall', 0.82);
0053   }
0054
0055   playGlassBreak(): void {
0056     this.play('glass', 0.72);
0057   }
0058 }
```

```
0059     playComplete(): void {
0060         this.play('complete', 0.78);
0061     }
0062
0063     playUi(): void {
0064         this.ensureAmbient();
0065         this.play('ui', 0.55);
0066     }
0067
0068     toggleSound(): boolean {
0069         this.setSoundEnabled(!this.soundEnabled);
0070         return this.soundEnabled;
0071     }
0072
0073     setSoundEnabled(enabled: boolean): void {
0074         this.soundEnabled = enabled;
0075         if (this.effectsSource) {
0076             this.effectsSource.volume = this.soundEnabled ? 1 : 0;
0077         }
0078         if (this.ambientSource) {
0079             this.ambientSource.volume = this.soundEnabled ? 0.32 : 0;
0080             if (this.soundEnabled && this.ambientRequested && this.ambientSource.clip) {
0081                 this.ambientSource.play();
0082             }
0083         }
0084     }
0085
0086     isSoundEnabled(): boolean {
0087         return this.soundEnabled;
0088     }
0089
0090     private createSource(name: string): AudioSource {
0091         const node = new Node(name);
0092         node.setParent(this.node);
0093         return node.addComponent(AudioSource);
0094     }
0095
0096     private loadEffect(path: string, name: EffectName): void {
0097         resources.load(path, AudioClip, (error, clip) => {
0098             if (error) {
0099                 console.warn(`Audio effect failed to load: ${path}`, error);
0100                 return;
0101             }
0102             this.effects.set(name, clip);
0103         });
0104     }
0105
0106     private ensureAmbient(): void {
0107         this.ambientRequested = true;
0108         if (this.soundEnabled && this.ambientSource?.clip && !this.ambientSource.playing) {
0109             this.ambientSource.play();
0110         }
0111     }
```

```
0111     }
0112
0113     private play(name: EffectName, volume: number): void {
0114         const clip = this.effects.get(name);
0115         if (clip) {
0116             this.effectsSource?.playOneShot(clip, volume);
0117         }
0118     }
0119 }
0001 import { sys } from 'cc';
0002 import {
0003     createDefaultReleaseSave,
0004     decodeReleaseSave,
0005     languageFromLocale,
0006     serializeReleaseSave,
0007     type LevelDefinition,
0008     type ReleaseSaveData,
0009 } from './shared/game/index';
0010
0011 const RELEASE_SAVE_KEY = 'cubic.release-save.v1';
0012
0013 export class ReleaseSaveRepository {
0014     constructor(private readonly levels: readonly LevelDefinition[]) {}
0015
0016     load(): ReleaseSaveData {
0017         try {
0018             return decodeReleaseSave(
0019                 sys.localStorage.getItem(RELEASE_SAVE_KEY),
0020                 this.levels,
0021                 languageFromLocale(sys.languageCode),
0022             );
0023         } catch (error) {
0024             console.warn('Release save could not be loaded.', error);
0025             return createDefaultReleaseSave(this.levels, languageFromLocale(sys.languageCode));
0026         }
0027     }
0028
0029     save(data: ReleaseSaveData): void {
0030         try {
0031             sys.localStorage.setItem(RELEASE_SAVE_KEY, serializeReleaseSave(data));
0032         } catch (error) {
0033             console.warn('Release save could not be written.', error);
0034         }
0035     }
0036
0037     reset(): ReleaseSaveData {
0038         const data = createDefaultReleaseSave(this.levels, languageFromLocale(sys.languageCode));
0039         this.save(data);
0040         return data;
0041     }
0042 }
0001 import { sys } from 'cc';
```

```

0002
0003 export interface Insets {
0004     readonly top: number;
0005     readonly right: number;
0006     readonly bottom: number;
0007     readonly left: number;
0008 }
0009
0010 export interface PlatformAdapter {
0011     readonly name: string;
0012     getSafeAreaInsets(): Insets;
0013     vibrateLight(): void;
0014     onLevelStarted(levelId: string): void;
0015     onLevelCompleted(levelId: string, steps: number): void;
0016 }
0017
0018 export class DefaultPlatformAdapter implements PlatformAdapter {
0019     readonly name: string = 'default';
0020
0021     getSafeAreaInsets(): Insets {
0022         return { top: 0, right: 0, bottom: 0, left: 0 };
0023     }
0024
0025     vibrateLight(): void {
0026         if (sys.isNative && sys.os === sys.OS.OPENHARMONY) {
0027             // A missing synchronous ArkTS reflection target blocks Cocos' script thread.
0028             return;
0029         }
0030
0031         globalThis.navigator?.vibrate?.(35);
0032     }
0033
0034     onLevelStarted(_levelId: string): void {
0035         // Intentionally empty. Native adapters can report lifecycle events later.
0036     }
0037
0038     onLevelCompleted(_levelId: string, _steps: number): void {
0039         // Intentionally empty. Native adapters can report lifecycle events later.
0040     }
0041 }
0042
0043 export function createPlatformAdapter(): PlatformAdapter {
0044     return new DefaultPlatformAdapter();
0045 }
0046
0001 import type { BlockState, GridCoord } from './types';
0002
0003 export function coordKey(coord: GridCoord): string {
0004     return `${coord.x},${coord.z}`;
0005 }
0006
0007 export function sameCoord(a: GridCoord, b: GridCoord): boolean {
0008     return a.x === b.x && a.z === b.z;

```

```
0009 }
0010
0011 export function occupiedCells(block: BlockState): GridCoord[] {
0012   const { x, z } = block.anchor;
0013
0014   if (block.orientation === 'standing') {
0015     return [{ x, z }];
0016   }
0017
0018   if (block.orientation === 'lying-x') {
0019     return [
0020       { x, z },
0021       { x: x + 1, z },
0022     ];
0023   }
0024
0025   return [
0026     { x, z },
0027     { x, z: z + 1 },
0028   ];
0029 }
0030
0031 export function boundsForCells(cells: readonly GridCoord[]): { minX: number; maxX: number; minZ: number; maxZ: number } {
0032   return cells.reduce(
0033     (bounds, cell) => ({
0034       minX: Math.min(bounds.minX, cell.x),
0035       maxX: Math.max(bounds.maxX, cell.x),
0036       minZ: Math.min(bounds.minZ, cell.z),
0037       maxZ: Math.max(bounds.maxZ, cell.z),
0038     }),
0039     { minX: Infinity, maxX: -Infinity, minZ: Infinity, maxZ: -Infinity },
0040   );
0041 }
0042
0043 import type { BlockState, Direction } from './types';
0044
0045 export function rollBlock(block: BlockState, direction: Direction): BlockState {
0046   const { x, z } = block.anchor;
0047
0048   if (block.orientation === 'standing') {
0049     switch (direction) {
0050       case 'left':
0051         return { anchor: { x: x - 2, z }, orientation: 'lying-x' };
0052       case 'right':
0053         return { anchor: { x: x + 1, z }, orientation: 'lying-x' };
0054       case 'up':
0055         return { anchor: { x, z: z - 2 }, orientation: 'lying-z' };
0056       case 'down':
0057         return { anchor: { x, z: z + 1 }, orientation: 'lying-z' };
0058     }
0059   }
0060
0061   if (block.orientation === 'lying-x') {
```

```
0020     switch (direction) {
0021         case 'left':
0022             return { anchor: { x: x - 1, z }, orientation: 'standing' };
0023         case 'right':
0024             return { anchor: { x: x + 2, z }, orientation: 'standing' };
0025         case 'up':
0026             return { anchor: { x, z: z - 1 }, orientation: 'lying-x' };
0027         case 'down':
0028             return { anchor: { x, z: z + 1 }, orientation: 'lying-x' };
0029     }
0030 }
0031
0032 switch (direction) {
0033     case 'left':
0034         return { anchor: { x: x - 1, z }, orientation: 'lying-z' };
0035     case 'right':
0036         return { anchor: { x: x + 1, z }, orientation: 'lying-z' };
0037     case 'up':
0038         return { anchor: { x, z: z - 1 }, orientation: 'standing' };
0039     case 'down':
0040         return { anchor: { x, z: z + 2 }, orientation: 'standing' };
0041 }
0042 }
0001 import { coordKey, occupiedCells, sameCoord } from './coords';
0002 import { assertValidLevel, LevelMap } from './level';
0003 import { rollBlock } from './movement';
0004 import type {
0005     BlockState,
0006     Direction,
0007     GridCoord,
0008     LevelDefinition,
0009     MoveResult,
0010     PuzzleState,
0011     SplitBlockState,
0012 } from './types';
0013
0014 export class PuzzleEngine {
0015     private readonly levelMap: LevelMap;
0016     private history: PuzzleState[] = [];
0017     private state: PuzzleState;
0018
0019     constructor(readonly level: LevelDefinition) {
0020         assertValidLevel(level);
0021         this.levelMap = new LevelMap(level);
0022         this.state = this.initialState();
0023     }
0024
0025     getState(): PuzzleState {
0026         return this.cloneState(this.state);
0027     }
0028
0029     getOccupiedCells(): GridCoord[] {
```



```
0030     if (this.state.split) {
0031         return this.state.split.cubes.map((cube) => ({ ...cube }));
0032     }
0033     return occupiedCells(this.state.block);
0034 }
0035
0036 move(direction: Direction): MoveResult {
0037     const previous = this.getState();
0038     if (this.state.failed || this.state.completed) {
0039         return this.invalidResult(previous, direction, 'Level is already resolved.');
```

```
0040     }
0041     return this.state.split
0042         ? this.moveSplit(previous, direction, this.state.split)
0043         : this.moveWhole(previous, direction);
0044 }
0045
0046 switchActiveCube(): PuzzleState {
0047     if (!this.state.split || this.state.failed || this.state.completed) {
0048         return this.getState();
0049     }
0050     const previous = this.getState();
0051     this.history.push(previous);
0052     this.state = {
0053         ...this.state,
0054         split: {
0055             cubes: this.cloneCubes(this.state.split.cubes),
0056             activeCube: this.state.split.activeCube === 0 ? 1 : 0,
0057         },
0058     };
0059     return this.getState();
0060 }
0061
0062 undo(): PuzzleState {
0063     const previous = this.history.pop();
0064     if (previous) this.state = previous;
0065     return this.getState();
0066 }
0067
0068 restart(): PuzzleState {
0069     this.history = [];
0070     this.state = this.initialState();
0071     return this.getState();
0072 }
0073
0074 canUndo(): boolean {
0075     return this.history.length > 0;
0076 }
0077
0078 private moveWhole(previous: PuzzleState, direction: Direction): MoveResult {
0079     const nextBlock = rollBlock(this.state.block, direction);
0080     const nextCells = occupiedCells(nextBlock);
0081     if (nextBlock.orientation === 'standing' && sameCoord(nextBlock.anchor, this.level.goal)) {
```

```

0082     return this.commit(previous, direction, {
0083         ...this.state,
0084         block: nextBlock,
0085         split: null,
0086         steps: this.state.steps + 1,
0087         completed: true,
0088     }, 'completed', nextCells, 'The block dropped into the goal hole.');
```

```

0089 }
0090
0091 if (!this.levelMap.supports(nextCells, this.state.bridgeStates)) {
0092     const supportedCells = this.levelMap.supportedCells(nextCells, this.state.bridgeStates);
0093     return this.commitFailure(previous, direction, {
0094         ...this.state,
0095         block: nextBlock,
0096         split: null,
0097         steps: this.state.steps + 1,
0098     }, nextCells, 'The block fell from the board.', supportedCells);
0099 }
0100
0101 if (nextBlock.orientation === 'standing'
0102     && this.levelMap.tileAt(nextBlock.anchor)?.type === 'fragile') {
0103     return this.commitFailure(previous, direction, {
0104         ...this.state,
0105         block: nextBlock,
0106         split: null,
0107         steps: this.state.steps + 1,
0108     }, nextCells, 'The standing block broke a fragile tile.', []);
0109 }
0110
0111 let bridgeStates = this.applyWholeSwitches(nextBlock, this.state.bridgeStates);
0112 const splitDefinition = nextBlock.orientation === 'standing'
0113     ? this.levelMap.splitAt(nextBlock.anchor)
0114     : undefined;
0115 const split = splitDefinition
0116     ? {
0117         cubes: this.cloneCubes(splitDefinition.destinations),
0118         activeCube: 0 as const,
0119     }
0120     : null;
0121 if (split && !this.levelMap.supports(split.cubes, bridgeStates)) {
0122     const supportedCells = this.levelMap.supportedCells(split.cubes, bridgeStates);
0123     return this.commitFailure(previous, direction, {
0124         ...this.state,
0125         block: nextBlock,
0126         split,
0127         bridgeStates,
0128         steps: this.state.steps + 1,
0129     }, split.cubes, 'A split cube destination is unsupported.', supportedCells);
0130 }
0131 return this.commit(previous, direction, {
0132     ...this.state,
0133     block: nextBlock,
```

```
0134     split,
0135     bridgeStates,
0136     steps: this.state.steps + 1,
0137   }, 'moved', split?.cubes ?? nextCells);
0138 }
0139
0140 private moveSplit(
0141   previous: PuzzleState,
0142   direction: Direction,
0143   split: SplitBlockState,
0144 ): MoveResult {
0145   const active = split.activeCube;
0146   const cubes = this.cloneCubes(split.cubes);
0147   cubes[active] = this.moveCube(cubes[active], direction);
0148   if (!this.levelMap.supports([cubes[active]], this.state.bridgeStates)) {
0149     const supportedCells = this.levelMap.supportedCells(cubes, this.state.bridgeStates);
0150     return this.commitFailure(previous, direction, {
0151       ...this.state,
0152       split: { cubes, activeCube: active },
0153       steps: this.state.steps + 1,
0154     }, cubes, 'The active split cube fell from the board.', supportedCells);
0155   }
0156
0157   const bridgeStates = this.applySoftSwitch(cubes[active], this.state.bridgeStates);
0158   const recombined = this.recombinedBlock(cubes);
0159   return this.commit(previous, direction, {
0160     ...this.state,
0161     block: recombined ?? this.state.block,
0162     split: recombined ? null : { cubes, activeCube: active },
0163     bridgeStates,
0164     steps: this.state.steps + 1,
0165   }, 'moved', recombined ? occupiedCells(recombined) : cubes);
0166 }
0167
0168 private applyWholeSwitches(
0169   block: BlockState,
0170   current: PuzzleState['bridgeStates'],
0171 ): PuzzleState['bridgeStates'] {
0172   let result = { ...current };
0173   const applied = new Set<string>();
0174   for (const cell of occupiedCells(block)) {
0175     const tile = this.levelMap.tileAt(cell);
0176     const qualifies = tile?.type === 'soft-switch'
0177       || (tile?.type === 'hard-switch' && block.orientation === 'standing');
0178     if (!qualifies || applied.has(coordKey(cell))) continue;
0179     result = this.applySwitch(cell, result);
0180     applied.add(coordKey(cell));
0181   }
0182   return result;
0183 }
0184
0185 private applySoftSwitch(
```

```
0186     cell: GridCoord,
0187     current: PuzzleState['bridgeStates'],
0188 ): PuzzleState['bridgeStates'] {
0189     return this.levelMap.tileAt(cell)?.type === 'soft-switch'
0190       ? this.applySwitch(cell, current)
0191       : { ...current };
0192 }
0193
0194 private applySwitch(
0195     cell: GridCoord,
0196     current: PuzzleState['bridgeStates'],
0197 ): PuzzleState['bridgeStates'] {
0198     const next = { ...current };
0199     for (const action of this.levelMap.switchAt(cell)?.actions ?? []) {
0200         if (action.mode === 'enable') next[action.bridgeId] = true;
0201         else if (action.mode === 'disable') next[action.bridgeId] = false;
0202         else next[action.bridgeId] = !next[action.bridgeId];
0203     }
0204     return next;
0205 }
0206
0207 private recombinedBlock(cubes: readonly [GridCoord, GridCoord]): BlockState | null {
0208     const [first, second] = cubes;
0209     if (first.z === second.z && Math.abs(first.x - second.x) === 1) {
0210         return {
0211             anchor: { x: Math.min(first.x, second.x), z: first.z },
0212             orientation: 'lying-x',
0213         };
0214     }
0215     if (first.x === second.x && Math.abs(first.z - second.z) === 1) {
0216         return {
0217             anchor: { x: first.x, z: Math.min(first.z, second.z) },
0218             orientation: 'lying-z',
0219         };
0220     }
0221     return null;
0222 }
0223
0224 private moveCube(cube: GridCoord, direction: Direction): GridCoord {
0225     if (direction === 'left') return { x: cube.x - 1, z: cube.z };
0226     if (direction === 'right') return { x: cube.x + 1, z: cube.z };
0227     if (direction === 'up') return { x: cube.x, z: cube.z - 1 };
0228     return { x: cube.x, z: cube.z + 1 };
0229 }
0230
0231 private commitFailure(
0232     previous: PuzzleState,
0233     direction: Direction,
0234     candidate: PuzzleState,
0235     cells: readonly GridCoord[],
0236     message: string,
0237     supportedCells: readonly GridCoord[],
```

```
0238 ): MoveResult {
0239     return this.commit(previous, direction, {
0240         ...candidate,
0241         failed: true,
0242         completed: false,
0243     }, 'fallen', cells, message, supportedCells);
0244 }
0245
0246 private commit(
0247     previous: PuzzleState,
0248     direction: Direction,
0249     candidate: PuzzleState,
0250     status: MoveResult['status'],
0251     cells: readonly GridCoord[],
0252     message?: string,
0253     supportedCells?: readonly GridCoord[],
0254 ): MoveResult {
0255     this.history.push(previous);
0256     this.state = this.cloneState(candidate);
0257     return {
0258         status,
0259         direction,
0260         previous,
0261         current: this.getState(),
0262         occupiedCells: cells.map((cell) => ({ ...cell })),
0263         supportedCells: supportedCells?.map((cell) => ({ ...cell })),
0264         message,
0265     };
0266 }
0267
0268 private invalidResult(previous: PuzzleState, direction: Direction, message: string): MoveResult {
0269     return {
0270         status: 'invalid',
0271         direction,
0272         previous,
0273         current: this.getState(),
0274         occupiedCells: this.getOccupiedCells(),
0275         message,
0276     };
0277 }
0278
0279 private initialState(): PuzzleState {
0280     const bridgeStates: Record<string, boolean> = {};
0281     for (const bridge of this.level.bridges ?? []) {
0282         bridgeStates[bridge.id] = bridge.initiallyActive;
0283     }
0284     return {
0285         levelId: this.level.id,
0286         block: {
0287             anchor: { ...this.level.start.anchor },
0288             orientation: this.level.start.orientation,
0289         },
```

```
0290     split: null,
0291     bridgeStates,
0292     steps: 0,
0293     failed: false,
0294     completed: false,
0295   };
0296 }
0297
0298 private cloneState(state: PuzzleState): PuzzleState {
0299   return {
0300     ...state,
0301     block: {
0302       anchor: { ...state.block.anchor },
0303       orientation: state.block.orientation,
0304     },
0305     split: state.split
0306     ? {
0307       cubes: this.cloneCubes(state.split.cubes),
0308       activeCube: state.split.activeCube,
0309     }
0310     : null,
0311     bridgeStates: { ...state.bridgeStates },
0312   };
0313 }
0314
0315 private cloneCubes(cubes: readonly [GridCoord, GridCoord]): [GridCoord, GridCoord] {
0316   return [{ ...cubes[0] }, { ...cubes[1] }];
0317 }
0318 }
0001 import { PuzzleEngine } from './PuzzleEngine';
0002 import { isGameLanguage } from './localization';
0003 import type { GameLanguage } from './localization';
0004 import type { LevelDefinition, PuzzleAction } from './types';
0005
0006 export const RELEASE_SAVE_VERSION = 1 as const;
0007
0008 export const tutorialTopicIds = [
0009   'movement',
0010   'goal',
0011   'fragile',
0012   'soft-switch',
0013   'hard-switch',
0014   'bridge',
0015   'split',
0016   'switch-cube',
0017   'recombine',
0018 ] as const;
0019
0020 export type TutorialTopicId = typeof tutorialTopicIds[number];
0021 export type StarRating = 1 | 2 | 3;
0022
0023 export interface SavedRun {
```

```
0024     readonly levelId: string;
0025     readonly actions: readonly PuzzleAction[];
0026     readonly elapsedSeconds: number;
0027 }
0028
0029 export interface ReleaseSaveData {
0030     readonly version: typeof RELEASE_SAVE_VERSION;
0031     readonly currentRun: SavedRun | null;
0032     readonly highestUnlockedLevelId: string;
0033     readonly soundEnabled: boolean;
0034     readonly language: GameLanguage;
0035     readonly acknowledgedTutorials: readonly TutorialTopicId[];
0036     readonly bestStarsByLevel: Readonly<Record<string, StarRating>>;
0037 }
0038
0039 export interface ResumedRun {
0040     readonly levelIndex: number;
0041     readonly engine: PuzzleEngine;
0042     readonly actions: readonly PuzzleAction[];
0043     readonly elapsedSeconds: number;
0044 }
0045
0046 const directions = new Set<PuzzleAction>(['up', 'down', 'left', 'right', 'switch-cube']);
0047 const tutorialTopics = new Set<string>(tutorialTopicIds);
0048
0049 export function createDefaultReleaseSave(
0050     levels: readonly LevelDefinition[],
0051     language: GameLanguage = 'en',
0052 ): ReleaseSaveData {
0053     if (levels.length === 0) throw new Error('At least one level is required for release save data.');
```

```
0054     return {
0055         version: RELEASE_SAVE_VERSION,
0056         currentRun: null,
0057         highestUnlockedLevelId: levels[0].id,
0058         soundEnabled: true,
0059         language,
0060         acknowledgedTutorials: [],
0061         bestStarsByLevel: {},
0062     };
0063 }
0064
0065 export function decodeReleaseSave(
0066     serialized: string | null | undefined,
0067     levels: readonly LevelDefinition[],
0068     fallbackLanguage: GameLanguage = 'en',
0069 ): ReleaseSaveData {
0070     if (!serialized) return createDefaultReleaseSave(levels, fallbackLanguage);
0071     try {
0072         return normalizeReleaseSave(JSON.parse(serialized), levels, fallbackLanguage);
0073     } catch {
0074         return createDefaultReleaseSave(levels, fallbackLanguage);
0075     }
```

```
0075     }
0076   }
0077
0078   export function normalizeReleaseSave(
0079     input: unknown,
0080     levels: readonly LevelDefinition[],
0081     fallbackLanguage: GameLanguage = 'en',
0082   ): ReleaseSaveData {
0083     const defaults = createDefaultReleaseSave(levels, fallbackLanguage);
0084     if (!isRecord(input)) return defaults;
0085
0086     const highestUnlockedLevelId = validLevelId(input.highestUnlockedLevelId, levels)
0087       ?? defaults.highestUnlockedLevelId;
0088     const soundEnabled = typeof input.soundEnabled === 'boolean'
0089       ? input.soundEnabled
0090       : defaults.soundEnabled;
0091     const language = isGameLanguage(input.language) ? input.language : defaults.language;
0092     const acknowledgedTutorials = Array.isArray(input.acknowledgedTutorials)
0093       ? Array.from(new Set(input.acknowledgedTutorials.filter(isTutorialTopicId)))
0094       : defaults.acknowledgedTutorials;
0095     const bestStarsByLevel = normalizeBestStars(input.bestStarsByLevel, levels);
0096     const currentRun = input.version === RELEASE_SAVE_VERSION
0097       ? normalizeSavedRun(input.currentRun, levels)
0098       : null;
0099
0100     return {
0101       version: RELEASE_SAVE_VERSION,
0102       currentRun,
0103       highestUnlockedLevelId,
0104       soundEnabled,
0105       language,
0106       acknowledgedTutorials,
0107       bestStarsByLevel,
0108     };
0109   }
0110
0111   export function serializeReleaseSave(save: ReleaseSaveData): string {
0112     return JSON.stringify(save);
0113   }
0114
0115   export function resumeSavedRun(
0116     save: ReleaseSaveData,
0117     levels: readonly LevelDefinition[],
0118   ): ResumedRun | null {
0119     const run = save.currentRun;
0120     if (!run) return null;
0121     const levelIndex = levels.findIndex((level) => level.id === run.levelId);
0122     if (levelIndex < 0) return null;
0123
0124     const engine = replayActions(levels[levelIndex], run.actions);
0125     if (!engine) return null;
```



```
0126     return {
0127         levelIndex,
0128         engine,
0129         actions: [...run.actions],
0130         elapsedSeconds: run.elapsedSeconds,
0131     };
0132 }
0133
0134 export function getHighestUnlockedIndex(
0135     save: ReleaseSaveData,
0136     levels: readonly LevelDefinition[],
0137 ): number {
0138     const index = levels.findIndex((level) => level.id === save.highestUnlockedLevelId);
0139     return index < 0 ? 0 : index;
0140 }
0141
0142 export function withUnlockedLevel(
0143     save: ReleaseSaveData,
0144     levelIndex: number,
0145     levels: readonly LevelDefinition[],
0146 ): ReleaseSaveData {
0147     const boundedIndex = Math.max(0, Math.min(Math.floor(levelIndex), levels.length - 1));
0148     const currentHighest = getHighestUnlockedIndex(save, levels);
0149     if (boundedIndex <= currentHighest) return save;
0150     return {
0151         ...save,
0152         highestUnlockedLevelId: levels[boundedIndex].id,
0153     };
0154 }
0155
0156 export function calculateStarRating(optimalMoves: number, playerMoves: number): StarRating {
0157     const optimal = Math.max(1, Math.floor(optimalMoves));
0158     const actual = Math.max(0, Math.floor(playerMoves));
0159     if (actual <= optimal) return 3;
0160     if (actual <= Math.ceil(optimal * 1.25)) return 2;
0161     return 1;
0162 }
0163
0164 export function getBestStarRating(save: ReleaseSaveData, levelId: string): StarRating | 0 {
0165     return save.bestStarsByLevel[levelId] ?? 0;
0166 }
0167
0168 export function withBestStarRating(
0169     save: ReleaseSaveData,
0170     levelId: string,
0171     rating: StarRating,
0172 ): ReleaseSaveData {
0173     if (getBestStarRating(save, levelId) >= rating) return save;
0174     return {
0175         ...save,
0176         bestStarsByLevel: {
```

```
0177     ...save.bestStarsByLevel,
0178     [levelId]: rating,
0179   },
0180 };
0181 }
0182
0183 export function withAcknowledgedTutorials(
0184   save: ReleaseSaveData,
0185   topicIds: readonly TutorialTopicId[],
0186 ): ReleaseSaveData {
0187   const acknowledged = new Set(save.acknowledgedTutorials);
0188   for (const topicId of topicIds) acknowledged.add(topicId);
0189   return {
0190     ...save,
0191     acknowledgedTutorials: Array.from(acknowledged),
0192   };
0193 }
0194
0195 export function resetCampaignProgress(
0196   save: ReleaseSaveData,
0197   levels: readonly LevelDefinition[],
0198 ): ReleaseSaveData {
0199   if (levels.length === 0) throw new Error('At least one level is required to reset campaign progress. ');
0200   return {
0201     ...save,
0202     currentRun: null,
0203     highestUnlockedLevelId: levels[0].id,
0204     acknowledgedTutorials: [],
0205     bestStarsByLevel: {},
0206   };
0207 }
0208
0209 function normalizeSavedRun(input: unknown, levels: readonly LevelDefinition[]): SavedRun | null {
0210   if (!isRecord(input)) return null;
0211   const levelId = validLevelId(input.levelId, levels);
0212   if (!levelId || !Array.isArray(input.actions)) return null;
0213   const actions = input.actions.filter(isPuzzleAction);
0214   if (actions.length !== input.actions.length) return null;
0215   const elapsedSeconds = typeof input.elapsedSeconds === 'number'
0216     && Number.isFinite(input.elapsedSeconds)
0217     && input.elapsedSeconds >= 0
0218     ? input.elapsedSeconds
0219     : 0;
0220   const level = levels.find((candidate) => candidate.id === levelId);
0221   if (!level || !replayActions(level, actions)) return null;
0222   return { levelId, actions, elapsedSeconds };
0223 }
0224
0225 function replayActions(
0226   level: LevelDefinition,
0227   actions: readonly PuzzleAction[],
```

```
0228 ): PuzzleEngine | null {
0229   const engine = new PuzzleEngine(level);
0230   for (const action of actions) {
0231     if (action === 'switch-cube') {
0232       if (!engine.getState().split) return null;
0233       engine.switchActiveCube();
0234       continue;
0235     }
0236     const result = engine.move(action);
0237     if (result.status !== 'moved') return null;
0238   }
0239   const state = engine.getState();
0240   return state.failed || state.completed ? null : engine;
0241 }
0242
0243 function validLevelId(input: unknown, levels: readonly LevelDefinition[]): string | null {
0244   if (typeof input !== 'string') return null;
0245   return levels.some((level) => level.id === input) ? input : null;
0246 }
0247
0248 function normalizeBestStars(
0249   input: unknown,
0250   levels: readonly LevelDefinition[],
0251 ): Readonly<Record<string, StarRating>> {
0252   if (!isRecord(input)) return {};
0253   const result: Record<string, StarRating> = {};
0254   for (const level of levels) {
0255     const rating = input[level.id];
0256     if (isStarRating(rating)) result[level.id] = rating;
0257   }
0258   return result;
0259 }
0260
0261 function isPuzzleAction(value: unknown): value is PuzzleAction {
0262   return typeof value === 'string' && directions.has(value as PuzzleAction);
0263 }
0264
0265 function isTutorialTopicId(value: unknown): value is TutorialTopicId {
0266   return typeof value === 'string' && tutorialTopics.has(value);
0267 }
0268
0269 function isStarRating(value: unknown): value is StarRating {
0270   return value === 1 || value === 2 || value === 3;
0271 }
0272
0273 function isRecord(value: unknown): value is Record<string, unknown> {
0274   return typeof value === 'object' && value !== null && !Array.isArray(value);
0275 }
0001 import { PuzzleEngine } from './PuzzleEngine';
0002 import type { LevelDefinition, PuzzleAction, PuzzleState } from './types';
0003
```

```
0004 export interface SolverResult {
0005   readonly solution: readonly PuzzleAction[] | null;
0006   readonly visitedStates: number;
0007   readonly exhausted: boolean;
0008 }
0009
0010 const directions: readonly PuzzleAction[] = ['up', 'down', 'left', 'right'];
0011
0012 export function findSolution(level: LevelDefinition, maxVisited = 20_000): SolverResult {
0013   const initial = new PuzzleEngine(level).getState();
0014   const queues: Array<Array<{ path: PuzzleAction[]; state: PuzzleState }>> = [
0015     [{ path: [], state: initial }],
0016   ];
0017   const bestMoveCounts = new Map<string, number>([[stateKey(initial), 0]]);
0018   let visitedStates = 1;
0019   let exhausted = true;
0020
0021   for (let moveCount = 0; moveCount < queues.length && visitedStates < maxVisited; moveCount += 1) {
0022     const queue = queues[moveCount] ?? [];
0023     for (let queueIndex = 0; queueIndex < queue.length && visitedStates < maxVisited; queueIndex += 1) {
0024       const { path, state: current } = queue[queueIndex];
0025       const actions = current.split ? [...directions, 'switch-cube' as const] : directions;
0026
0027       for (const action of actions) {
0028         const candidate = new PuzzleEngine(level);
0029         replayInto(candidate, path);
0030         if (action === 'switch-cube') candidate.switchActiveCube();
0031         else candidate.move(action);
0032
0033         const state = candidate.getState();
0034         if (state.failed) continue;
0035         const nextPath = [...path, action];
0036         if (state.completed) {
0037           return { solution: nextPath, visitedStates, exhausted: false };
0038         }
0039         const nextMoveCount = moveCount + (action === 'switch-cube' ? 0 : 1);
0040         const key = stateKey(state);
0041         if ((bestMoveCounts.get(key) ?? Number.POSITIVE_INFINITY) <= nextMoveCount) continue;
0042         bestMoveCounts.set(key, nextMoveCount);
0043         visitedStates += 1;
0044         if (!queues[nextMoveCount]) queues[nextMoveCount] = [];
0045         queues[nextMoveCount].push({ path: nextPath, state });
0046         if (visitedStates >= maxVisited) {
0047           exhausted = false;
0048           break;
0049         }
0050       }
0051     }
0052   }
0053
0054   return {
```

```
0055     solution: null,
0056     visitedStates,
0057     exhausted,
0058   };
0059 }
0060
0061 function replayInto(engine: PuzzleEngine, path: readonly PuzzleAction[]): void {
0062   for (const action of path) {
0063     if (action === 'switch-cube') engine.switchActiveCube();
0064     else engine.move(action);
0065   }
0066 }
0067
0068 function stateKey(state: PuzzleState): string {
0069   const block = `${state.block.anchor.x},${state.block.anchor.z},${state.block.orientation}`;
0070   const split = state.split
0071     ?
0072     : '-';
0073   const bridges = Object.keys(state.bridgeStates)
0074     .sort()
0075     .map((id) => `${id}:${state.bridgeStates[id] ? 1 : 0}`)
0076     .join(';');
0077   return `${block}|${split}|${bridges}`;
0078 }
0079
0080 import type { LevelDefinition } from './types';
0081 import type { TutorialTopicId } from './releaseSave';
0082
0083 export interface OnboardingTopic {
0084   readonly id: TutorialTopicId;
0085   readonly title: string;
0086   readonly symbol: string;
0087   readonly body: string;
0088 }
0089
0090 export const onboardingTopics: readonly OnboardingTopic[] = [
0091   {
0092     id: 'movement',
0093     title: 'ROLL THE BLOCK',
0094     symbol: ' UP\nLEFT RIGHT\n DOWN',
0095     body: 'Swipe up, down, left, or right. Each swipe rolls the block one step in that screen direction.',
0096   },
0097   {
0098     id: 'goal',
0099     title: 'ENTER THE GOAL',
0100     symbol: 'UPRIGHT\n +\n HOLE',
0101     body: 'Finish the stage by standing the whole block upright over the dark hole so it can drop inside.',
0102   },
0103   {
0104     id: 'fragile',
0105     title: 'FRAGILE TILE',
0106     symbol: 'o\nGLASS',
0107   },
0108 ]
```

```

0028     body: 'You may cross this translucent tile while lying down. Standing upright on it will break it.',
0029   },
0030   {
0031     id: 'soft-switch',
0032     title: 'SOFT SWITCH',
0033     symbol: 'O',
0034     body: 'Any part of the whole block or either split cube can press it. A switch may open, close, or toggle its linked bridges.',
0035   },
0036   {
0037     id: 'hard-switch',
0038     title: 'HARD SWITCH',
0039     symbol: 'X',
0040     body: 'An X-marked hard switch responds only when the unsplit whole block stands upright on it.',
0041   },
0042   {
0043     id: 'bridge',
0044     title: 'CONTROLLED BRIDGE',
0045     symbol: '⚓\n⚓\n⚓',
0046     body: 'Switches can enable, disable, or toggle connected bridge spans. Check the route after every press.',
0047   },
0048   {
0049     id: 'split',
0050     title: 'SPLIT TILE',
0051     symbol: '◀|▶',
0052     body: 'Stand upright on the split tile to divide the block into two independently positioned cubes.',
0053   },
0054   {
0055     id: 'switch-cube',
0056     title: 'SWITCH ACTIVE CUBE',
0057     symbol: 'A / B',
0058     body: 'Only the highlighted cube moves. Use Switch Block at the upper left to select the other cube.',
0059   },
0060   {
0061     id: 'recombine',
0062     title: 'RECOMBINE',
0063     symbol: 'A + B\nBLOCK',
0064     body: 'Move the two cubes onto orthogonally adjacent tiles. They will automatically reform the whole block.',
0065   },
0066 ];
0067
0068 export function onboardingTopicsForLevel(level: LevelDefinition): OnboardingTopic[] {
0069   const included = new Set<TutorialTopicId>(['movement', 'goal']);
0070   const tileTypes = new Set<LevelTile>(level.tiles.map((tile) => tile.type));
0071   if (tileTypes.has('fragile')) included.add('fragile');
0072   if (tileTypes.has('soft-switch')) included.add('soft-switch');
0073   if (tileTypes.has('hard-switch')) included.add('hard-switch');
0074   if ((level.bridges?.length ?? 0) > 0) included.add('bridge');
0075   if ((level.splits?.length ?? 0) > 0 || tileTypes.has('split')) {
0076     included.add('split');
0077     included.add('switch-cube');
0078     included.add('recombine');

```

```

0079     }
0080     return onboardingTopics.filter((topic) => included.has(topic.id));
0081 }
0082
0083 export function pendingOnboardingTopics(
0084     level: LevelDefinition,
0085     acknowledged: readonly TutorialTopicId[],
0086 ): OnboardingTopic[] {
0087     const acknowledgedSet = new Set<TutorialTopicId>(acknowledged);
0088     return onboardingTopicsForLevel(level).filter((topic) => !acknowledgedSet.has(topic.id));
0089 }
0090
0091 import {
0092     _decorator,
0093     Color,
0094     Component,
0095     instantiate,
0096     Material,
0097     MeshRenderer,
0098     Node,
0099     primitives,
0100     Quat,
0101     tween,
0102     Tween,
0103     utils,
0104     Vec3,
0105 } from 'cc';
0106 import { occupiedCells } from './shared/game/index';
0107 import type { LevelDefinition, PuzzleState, TileDefinition } from './shared/game/index';
0108
0109 const { ccclass, property } = _decorator;
0110
0111 @ccclass('BoardRenderer')
0112 export class BoardRenderer extends Component {
0113     @property(Node)
0114     tilePrefab: Node | null = null;
0115
0116     @property
0117     tileSize = 1;
0118
0119     @property(Material)
0120     litBaseMaterial: Material | null = null;
0121
0122     private spawnedTiles: Node[] = [];
0123     private readonly bridgeNodes = new Map<string, Node[]>();
0124     private readonly bridgeStates = new Map<string, boolean>();
0125     private readonly switchNodes = new Map<string, Node>();
0126     private readonly fragileNodes = new Map<string, Node>();
0127     private readonly breakEffectNodes = new Set<Node>();
0128     private readonly breakTweenTargets = new Set<object>();
0129     private readonly materials = new Map<string, Material>();
0130 }

```

```
0041 render(level: LevelDefinition): void {
0042     this.clear();
0043     for (const tile of level.tiles) {
0044         const tileNode = this.createTile(tile);
0045         tileNode.setParent(this.node);
0046         tileNode.setPosition(new Vec3(tile.x * this.tileSize, 0, tile.z * this.tileSize));
0047         this.spawnedTiles.push(tileNode);
0048         if (tile.type === 'fragile') this.fragileNodes.set(this.tileKey(tile.x, tile.z), tileNode);
0049         if (tile.type === 'soft-switch' || tile.type === 'hard-switch') {
0050             this.switchNodes.set(this.tileKey(tile.x, tile.z), tileNode);
0051         }
0052     }
0053     for (const bridge of level.bridges ?? []) {
0054         const nodes: Node[] = [];
0055         for (const cell of bridge.cells) {
0056             const bridgeNode = this.createBridgeTile(bridge.id);
0057             bridgeNode.setParent(this.node);
0058             bridgeNode.setPosition(new Vec3(cell.x * this.tileSize, 0, cell.z * this.tileSize));
0059             this.spawnedTiles.push(bridgeNode);
0060             nodes.push(bridgeNode);
0061         }
0062         this.bridgeNodes.set(bridge.id, nodes);
0063     }
0064
0065     const goalHole = this.createGoalHole();
0066     goalHole.setParent(this.node);
0067     goalHole.setPosition(new Vec3(level.goal.x * this.tileSize, 0, level.goal.z * this.tileSize));
0068     this.spawnedTiles.push(goalHole);
0069 }
0070
0071 applyState(state: PuzzleState): void {
0072     let bridgeChanged = false;
0073     for (const [bridgeId, nodes] of this.bridgeNodes) {
0074         const active = state.bridgeStates[bridgeId] === true;
0075         const previous = this.bridgeStates.get(bridgeId);
0076         this.bridgeStates.set(bridgeId, active);
0077         if (previous === undefined) {
0078             for (const node of nodes) this.snapBridge(node, active);
0079         } else if (previous !== active) {
0080             bridgeChanged = true;
0081             for (const node of nodes) this.animateBridge(node, bridgeId, active);
0082         }
0083     }
0084     if (bridgeChanged) this.animatePressedSwitch(state);
0085 }
0086
0087 playFragileBreak(coord: { x: number; z: number }): void {
0088     const tile = this.fragileNodes.get(this.tileKey(coord.x, coord.z));
0089     if (!tile?.active) return;
0090     tile.active = false;
0091 }
```



```
0092     for (let index = 0; index < 9; index += 1) {
0093         const column = index % 3;
0094         const row = Math.floor(index / 3);
0095         const offsetX = (column - 1) * 0.29;
0096         const offsetZ = (row - 1) * 0.29;
0097         const width = 0.22 + (index % 2) * 0.05;
0098         const depth = 0.21 + ((index + 1) % 3) * 0.025;
0099         const shard = this.addBox(
0100             this.node,
0101             `GlassShard${index + 1}`,
0102             new Vec3(width, 0.04, depth),
0103             new Vec3(tile.position.x + offsetX, 0.12, tile.position.z + offsetZ),
0104             new Color(183, 226, 235, 145),
0105             this.getLitMaterial(new Color(183, 226, 235, 145), 0.14),
0106         );
0107         shard.setRotationFromEuler(0, index * 19, 0);
0108         const renderer = shard.getComponent(MeshRenderer);
0109         if (renderer) renderer.shadowCastingMode = MeshRenderer.ShadowCastingMode.ON;
0110         this.breakEffectNodes.add(shard);
0111         this.animateGlassShard(shard, index, offsetX, offsetZ);
0112     }
0113 }
0114
0115 clear(): void {
0116     for (const target of this.breakTweenTargets) Tween.stopAllByTarget(target);
0117     this.breakTweenTargets.clear();
0118     for (const effect of this.breakEffectNodes) effect.destroy();
0119     this.breakEffectNodes.clear();
0120     for (const tile of this.spawnedTiles) tile.destroy();
0121     this.spawnedTiles = [];
0122     this.bridgeNodes.clear();
0123     this.bridgeStates.clear();
0124     this.switchNodes.clear();
0125     this.fragileNodes.clear();
0126 }
0127
0128 private createTile(tile: TileDefinition): Node {
0129     const tileNode = this.tilePrefab ? instantiate(this.tilePrefab) : new Node(`tile-${tile.x}-${tile.z}`);
0130     tileNode.name = `tile-${tile.x}-${tile.z}`;
0131     if (!this.tilePrefab) {
0132         const sideColor = tile.type === 'fragile'
0133             ? new Color(112, 172, 187, 92)
0134             : new Color(157, 162, 172, 255);
0135         this.addBase(tileNode, sideColor, tile.type === 'fragile' ? 0.18 : 0.88);
0136         this.createTileTop(tileNode, tile);
0137     }
0138     return tileNode;
0139 }
0140
0141 private createTileTop(tileNode: Node, tile: TileDefinition): void {
0142     if (tile.type === 'fragile') {
```

```

0143     const glassColor = new Color(184, 226, 234, 112);
0144     const edgeColor = new Color(218, 244, 248, 185);
0145     const glassMaterial = this.getLitMaterial(glassColor, 0.12);
0146     const edgeMaterial = this.getLitMaterial(edgeColor, 0.16);
0147     this.addBox(tileNode, 'GlassTop', new Vec3(0.984, 0.035, 0.984), new Vec3(0, 0.105, 0), glassColor, glassMaterial);
0148     this.addBox(tileNode, 'GlassEdgeNorth', new Vec3(0.94, 0.025, 0.025), new Vec3(0, 0.135, -0.47), edgeColor, edgeMaterial);
0149     this.addBox(tileNode, 'GlassEdgeSouth', new Vec3(0.94, 0.025, 0.025), new Vec3(0, 0.135, 0.47), edgeColor, edgeMaterial);
0150     this.addBox(tileNode, 'GlassEdgeWest', new Vec3(0.025, 0.025, 0.94), new Vec3(-0.47, 0.135, 0), edgeColor, edgeMaterial);
0151     this.addBox(tileNode, 'GlassEdgeEast', new Vec3(0.025, 0.025, 0.94), new Vec3(0.47, 0.135, 0), edgeColor, edgeMaterial);
0152     return;
0153 }
0154
0155 const topColor = new Color(238, 239, 242, 255);
0156 this.addBox(
0157     tileNode,
0158     'StoneTop',
0159     new Vec3(0.984, 0.035, 0.984),
0160     new Vec3(0, 0.105, 0),
0161     topColor,
0162 );
0163
0164 if (tile.type === 'soft-switch') {
0165     this.addCylinder(tileNode, 'SoftSwitchOuter', 0.24, 0.045, new Vec3(0, 0.15, 0), new Color(39, 127, 114, 255));
0166     this.addCylinder(tileNode, 'SoftSwitchInner', 0.15, 0.05, new Vec3(0, 0.17, 0), topColor);
0167 } else if (tile.type === 'hard-switch') {
0168     const first = this.addBox(tileNode, 'HardSwitchA', new Vec3(0.58, 0.045, 0.1), new Vec3(0, 0.16, 0), new Color(173, 94, 35,
255));
0169     const second = this.addBox(tileNode, 'HardSwitchB', new Vec3(0.58, 0.045, 0.1), new Vec3(0, 0.162, 0), new Color(173, 94, 35,
255));
0170     first.setRotationFromEuler(0, 45, 0);
0171     second.setRotationFromEuler(0, -45, 0);
0172 } else if (tile.type === 'split') {
0173     const markerColor = new Color(102, 91, 158, 255);
0174     const left = this.addCylinder(tileNode, 'SplitDoorLeft', 0.21, 0.04, new Vec3(-0.22, 0.155, 0), markerColor, 3);
0175     const right = this.addCylinder(tileNode, 'SplitDoorRight', 0.21, 0.04, new Vec3(0.22, 0.155, 0), markerColor, 3);
0176     left.setRotationFromEuler(0, -90, 0);
0177     right.setRotationFromEuler(0, 90, 0);
0178     this.addBox(tileNode, 'SplitDoorDivider', new Vec3(0.045, 0.04, 0.5), new Vec3(0, 0.155, 0), markerColor);
0179 }
0180 }
0181
0182 private createBridgeTile(id: string): Node {
0183     const node = new Node(`bridge-${id}`);
0184     this.addBase(node, new Color(40, 80, 91, 255));
0185     this.addBox(node, 'BridgeTop', new Vec3(0.984, 0.045, 0.984), new Vec3(0, 0.11, 0), new Color(78, 151, 159, 255));
0186     return node;
0187 }
0188
0189 private snapBridge(node: Node, active: boolean): void {
0190     Tween.stopAllByTarget(node);
0191     node.setScale(1, 1, 1);
0192     node.setPosition(node.position.x, active ? 0 : -0.26, node.position.z);
0193     node.active = active;

```

```
0194     }
0195
0196 private animateBridge(node: Node, bridgeId: string, active: boolean): void {
0197     Tween.stopAllByTarget(node);
0198     const x = node.position.x;
0199     const z = node.position.z;
0200     node.active = true;
0201     if (active) {
0202         node.setPosition(x, Math.min(node.position.y, -0.22), z);
0203         node.setScale(0.94, 0.76, 0.94);
0204         tween(node)
0205             .to(0.2, {
0206                 position: new Vec3(x, 0.055, z),
0207                 scale: new Vec3(1.035, 1.08, 1.035),
0208             }, { easing: 'quadOut' })
0209             .to(0.11, {
0210                 position: new Vec3(x, 0, z),
0211                 scale: new Vec3(1, 1, 1),
0212             }, { easing: 'quadInOut' })
0213             .start();
0214         return;
0215     }
0216
0217     tween(node)
0218         .to(0.08, {
0219             position: new Vec3(x, 0.035, z),
0220             scale: new Vec3(1.025, 1.04, 1.025),
0221         }, { easing: 'quadOut' })
0222         .to(0.22, {
0223             position: new Vec3(x, -0.26, z),
0224             scale: new Vec3(0.94, 0.76, 0.94),
0225         }, { easing: 'quadIn' })
0226         .call(() => {
0227             if (this.bridgeStates.get(bridgeId) === false) node.active = false;
0228         })
0229         .start();
0230 }
0231
0232 private animatePressedSwitch(state: PuzzleState): void {
0233     const cells = state.split ? state.split.cubes : occupiedCells(state.block);
0234     for (const cell of cells) {
0235         const switchNode = this.switchNodes.get(this.tileKey(cell.x, cell.z));
0236         if (!switchNode) continue;
0237         const parts = [
0238             switchNode.getChildByName('SoftSwitchOuter'),
0239             switchNode.getChildByName('SoftSwitchInner'),
0240             switchNode.getChildByName('HardSwitchA'),
0241             switchNode.getChildByName('HardSwitchB'),
0242         ];
0243         for (const part of parts) {
0244             if (!part) continue;
```

```
0245     Tween.stopAllByTarget(part);
0246     const rest = part.position.clone();
0247     tween(part)
0248       .to(0.07, {
0249         position: new Vec3(rest.x, rest.y - 0.035, rest.z),
0250         scale: new Vec3(1.06, 0.72, 1.06),
0251       }, { easing: 'quadOut' })
0252       .to(0.15, {
0253         position: rest,
0254         scale: new Vec3(1, 1, 1),
0255       }, { easing: 'quadInOut' })
0256       .start();
0257   }
0258 }
0259 }
0260
0261 private addBase(parent: Node, color: Color, roughness = 0.88): void {
0262   const renderer = parent.addComponent(MeshRenderer);
0263   renderer.mesh = utils.createMesh(primitives.box({ width: 1, height: 0.2, length: 1 }));
0264   renderer.setMaterial(this.getLitMaterial(color, roughness), 0);
0265   this.configureShadowReceiver(renderer);
0266 }
0267
0268 private createGoalHole(): Node {
0269   const hole = new Node('GoalHole');
0270   const shaftColor = new Color(18, 19, 23, 255);
0271   const floorColor = new Color(2, 3, 5, 255);
0272   const rimColor = new Color(105, 108, 116, 255);
0273   this.addGoalBox(hole, 'ShaftNorth', new Vec3(0.88, 0.7, 0.035), new Vec3(0, -0.25, -0.45), shaftColor);
0274   this.addGoalBox(hole, 'ShaftSouth', new Vec3(0.88, 0.7, 0.035), new Vec3(0, -0.25, 0.45), shaftColor);
0275   this.addGoalBox(hole, 'ShaftWest', new Vec3(0.035, 0.7, 0.88), new Vec3(-0.45, -0.25, 0), shaftColor);
0276   this.addGoalBox(hole, 'ShaftEast', new Vec3(0.035, 0.7, 0.88), new Vec3(0.45, -0.25, 0), shaftColor);
0277   this.addGoalBox(hole, 'ShaftFloor', new Vec3(0.86, 0.025, 0.86), new Vec3(0, -0.62, 0), floorColor);
0278   this.addGoalBox(hole, 'RimNorth', new Vec3(0.94, 0.035, 0.05), new Vec3(0, 0.03, -0.47), rimColor);
0279   this.addGoalBox(hole, 'RimSouth', new Vec3(0.94, 0.035, 0.05), new Vec3(0, 0.03, 0.47), rimColor);
0280   this.addGoalBox(hole, 'RimWest', new Vec3(0.05, 0.035, 0.84), new Vec3(-0.47, 0.03, 0), rimColor);
0281   this.addGoalBox(hole, 'RimEast', new Vec3(0.05, 0.035, 0.84), new Vec3(0.47, 0.03, 0), rimColor);
0282   return hole;
0283 }
0284
0285 private addGoalBox(
0286   parent: Node,
0287   name: string,
0288   size: Vec3,
0289   position: Vec3,
0290   color: Color,
0291 ): Node {
0292   const node = this.addBox(parent, name, size, position, color);
0293   const renderer = node.getComponent(MeshRenderer);
0294   if (renderer) renderer.shadowCastingMode = MeshRenderer.ShadowCastingMode.ON;
0295   return node;
```

```
0296     }
0297
0298     private addBox(parent: Node, name: string, size: Vec3, position: Vec3, color: Color, material?: Material): Node {
0299         const node = new Node(name);
0300         node.setParent(parent);
0301         node.setPosition(position);
0302         const renderer = node.addComponent(MeshRenderer);
0303         renderer.mesh = utils.createMesh(primitives.box({ width: size.x, height: size.y, length: size.z }));
0304         renderer.setMaterial(material ?? this.getLitMaterial(color), 0);
0305         this.configureShadowReceiver(renderer);
0306         return node;
0307     }
0308
0309     private addCylinder(
0310         parent: Node,
0311         name: string,
0312         radius: number,
0313         height: number,
0314         position: Vec3,
0315         color: Color,
0316         radialSegments = 20,
0317     ): Node {
0318         const node = new Node(name);
0319         node.setParent(parent);
0320         node.setPosition(position);
0321         const renderer = node.addComponent(MeshRenderer);
0322         renderer.mesh = utils.createMesh(primitives.cylinder(radius, radius, height, { radialSegments }));
0323         renderer.setMaterial(this.getLitMaterial(color), 0);
0324         this.configureShadowReceiver(renderer);
0325         return node;
0326     }
0327
0328     private getLitMaterial(color: Color, roughness = 0.88): Material {
0329         const key = `${color.r}-${color.g}-${color.b}-${color.a}-${roughness}`;
0330         const existing = this.materials.get(key);
0331         if (existing) return existing;
0332         const baseMaterial = this.litBaseMaterial;
0333         if (!baseMaterial) throw new Error('Board lit material is not assigned');
0334         const material = new Material();
0335         material.copy(baseMaterial, { technique: color.a < 255 ? 1 : 0 });
0336         material.setProperty('mainColor', color);
0337         material.setProperty('roughness', roughness);
0338         material.setProperty('metallic', 0);
0339         this.materials.set(key, material);
0340         return material;
0341     }
0342
0343     private animateGlassShard(
0344         shard: Node,
0345         index: number,
0346         offsetX: number,
```

```
0347     offsetZ: number,
0348 ): void {
0349     const start = shard.position.clone();
0350     const radialX = offsetX * 1.25 + ((index % 2 === 0 ? 1 : -1) * 0.08);
0351     const radialZ = offsetZ * 1.25 + ((index % 3 === 0 ? -1 : 1) * 0.07);
0352     const lift = 0.22 + (index % 3) * 0.055;
0353     const progress = { value: 0 };
0354     const position = new Vec3();
0355     const rotation = new Quat();
0356     this.breakTweenTargets.add(progress);
0357     tween(progress)
0358       .to(0.58, { value: 1 }, {
0359         easing: 'linear',
0360         onUpdate: () => {
0361           const phase = progress.value;
0362           position.set(
0363             start.x + radialX * phase,
0364             start.y + lift * Math.sin(Math.PI * phase) - 1.75 * phase * phase,
0365             start.z + radialZ * phase,
0366           );
0367           shard.setPosition(position);
0368           Quat.fromEuler(
0369             rotation,
0370             (index + 2) * 83 * phase,
0371             index * 19 + (index + 1) * 61 * phase,
0372             (index + 1) * 47 * phase,
0373           );
0374           shard.setRotation(rotation);
0375           const scale = phase < 0.72 ? 1 : Math.max(0, 1 - (phase - 0.72) / 0.28);
0376           shard.setScale(scale, scale, scale);
0377         },
0378       })
0379       .call(() => {
0380         this.breakTweenTargets.delete(progress);
0381         this.breakEffectNodes.delete(shard);
0382         shard.destroy();
0383       })
0384       .start();
0385   }
0386
0387   private tileKey(x: number, z: number): string {
0388     return `${x},${z}`;
0389   }
0390
0391   private configureShadowReceiver(renderer: MeshRenderer): void {
0392     renderer.receiveShadow = MeshRenderer.ShadowReceivingMode.ON;
0393     renderer.shadowBias = 0.0005;
0394     renderer.shadowNormalBias = 0.02;
0395   }
0396
0397 }
```

```
0001 import { _decorator, Component, Node, Quat, tween, Tween, Vec3 } from 'cc';
0002 import type { BlockState, Direction, MoveResult, PuzzleState } from './shared/game/index';
0003
0004 const { ccclass, property } = _decorator;
0005
0006 interface FallEdge {
0007     readonly direction: Direction;
0008     readonly pivot: Vec3;
0009 }
0010
0011 @ccclass('BlockPresenter')
0012 export class BlockPresenter extends Component {
0013     @property
0014     tileSize = 1;
0015
0016     @property
0017     rollDuration = 0.22;
0018
0019     private busy = false;
0020     private wholeNode: Node | null = null;
0021     private cubeNodes: [Node | null, Node | null] = [null, null];
0022     private activeCubeIndex: 0 | 1 = 0;
0023     private motionTweenTarget: object | null = null;
0024
0025     start(): void {
0026         this.resolveNodes();
0027     }
0028
0029     isBusy(): boolean {
0030         return this.busy;
0031     }
0032
0033     startAttract(state: PuzzleState): void {
0034         this.stopAttract();
0035         this.snapTo(state);
0036     }
0037
0038     stopAttract(): void {
0039         this.resolveNodes();
0040         if (this.wholeNode) Tween.stopAllByTarget(this.wholeNode);
0041         if (this.motionTweenTarget) Tween.stopAllByTarget(this.motionTweenTarget);
0042         this.motionTweenTarget = null;
0043         this.busy = false;
0044     }
0045
0046     snapTo(state: PuzzleState): void {
0047         this.resolveNodes();
0048         if (state.split) {
0049             this.activeCubeIndex = state.split.activeCube;
0050             if (this.wholeNode) this.wholeNode.active = false;
0051             for (const index of [0, 1] as const) {
0052                 const cube = this.cubeNodes[index];
0053                 if (!cube) continue;
```

```
0054         cube.active = true;
0055         cube.setPosition(this.cubePosition(state.split.cubes[index]));
0056         const isActive = state.split.activeCube === index;
0057         const scale = isActive ? 1.08 : 0.96;
0058         cube.setScale(scale, scale, scale);
0059         const marker = cube.getChildByName('SelectionMarker');
0060         if (marker) marker.active = isActive;
0061     }
0062     return;
0063 }
0064
0065 for (const cube of this.cubeNodes) {
0066     if (cube) cube.active = false;
0067 }
0068 if (!this.wholeNode) return;
0069 this.wholeNode.active = true;
0070 this.wholeNode.setScale(Vec3.ONE);
0071 this.wholeNode.setPosition(this.positionFor(state.block));
0072 this.wholeNode.setRotation(this.rotationFor(state.block));
0073 }
0074
0075 playMove(result: MoveResult, onComplete: () => void, durationScale = 1): void {
0076     this.resolveNodes();
0077     if (result.previous.split) {
0078         this.playSplitMove(result, onComplete, durationScale);
0079         return;
0080     }
0081     this.playWholeMove(result, onComplete, durationScale);
0082 }
0083
0084 playFall(result: MoveResult, onComplete: () => void): void {
0085     this.busy = true;
0086     const node = this.activeVisibleNode();
0087     if (!node) {
0088         this.busy = false;
0089         onComplete();
0090         return;
0091     }
0092
0093     const startPosition = node.position.clone();
0094     const startRotation = node.rotation.clone();
0095     const edge = this.partialSupportEdge(result);
0096     if (!edge) {
0097         this.playUnsupportedFall(node, startPosition, onComplete);
0098         return;
0099     }
0100
0101     const direction = this.directionVector(edge.direction);
0102     const axis = this.rotationAxis(edge.direction);
0103     const relative = startPosition.clone().subtract(edge.pivot);
0104     const tipAngle = Math.PI * 0.4;
0105     const tipRotation = new Quat();
0106     Quat.fromAxisAngle(tipRotation, axis, tipAngle);
```



```
0107     const tipOffset = new Vec3();
0108     Vec3.transformQuat(tipOffset, relative, tipRotation);
0109     const tipPosition = edge.pivot.clone().add(tipOffset);
0110     const progress = { value: 0 };
0111     const rotationDelta = new Quat();
0112     const rotation = new Quat();
0113     const position = new Vec3();
0114
0115     tween(progress)
0116       .to(0.68, { value: 1 }, {
0117         easing: 'linear',
0118         onUpdate: () => {
0119           const tipPhase = 0.42;
0120           if (progress.value <= tipPhase) {
0121             const phase = progress.value / tipPhase;
0122             const eased = phase * phase;
0123             Quat.fromAxisAngle(rotationDelta, axis, tipAngle * eased);
0124             Vec3.transformQuat(position, relative, rotationDelta);
0125             position.add(edge.pivot);
0126           } else {
0127             const phase = (progress.value - tipPhase) / (1 - tipPhase);
0128             position.set(tipPosition);
0129             Vec3.scaleAndAdd(position, position, direction, 0.85 * phase);
0130             position.y -= 3.2 * phase * phase;
0131             Quat.fromAxisAngle(rotationDelta, axis, tipAngle + Math.PI * 0.95 * phase);
0132           }
0133           node.setPosition(position);
0134           Quat.multiply(rotation, rotationDelta, startRotation);
0135           node.setRotation(rotation);
0136         },
0137       })
0138       .call(() => this.finishFall(onComplete))
0139       .start();
0140   }
0141
0142   private playUnsupportedFall(
0143     node: Node,
0144     startPosition: Vec3,
0145     onComplete: () => void,
0146   ): void {
0147     const progress = { value: 0 };
0148     const position = new Vec3();
0149     tween(progress)
0150       .to(0.48, { value: 1 }, {
0151         easing: 'linear',
0152         onUpdate: () => {
0153           const phase = progress.value;
0154           position.set(startPosition);
0155           position.y -= 3.5 * phase * phase;
0156           node.setPosition(position);
0157         },
0158       })
0159       .call(() => this.finishFall(onComplete))
```

```
0160     .start();
0161 }
0162
0163 private partialSupportEdge(result: MoveResult): FallEdge | null {
0164     if (result.current.split || result.occupiedCells.length !== 2) return null;
0165     const supportedCells = result.supportedCells ?? [];
0166     if (supportedCells.length !== 1) return null;
0167     const supported = supportedCells[0];
0168     const unsupported = result.occupiedCells.find((cell) =>
0169         cell.x !== supported.x || cell.z !== supported.z);
0170     if (!unsupported) return null;
0171
0172     let direction: Direction;
0173     if (unsupported.x < supported.x) direction = 'left';
0174     else if (unsupported.x > supported.x) direction = 'right';
0175     else if (unsupported.z < supported.z) direction = 'up';
0176     else if (unsupported.z > supported.z) direction = 'down';
0177     else return null;
0178     return {
0179         direction,
0180         pivot: new Vec3(
0181             (supported.x + unsupported.x) * this.tileSize * 0.5,
0182             0,
0183             (supported.z + unsupported.z) * this.tileSize * 0.5,
0184         ),
0185     };
0186 }
0187
0188 private finishFall(onComplete: () => void): void {
0189     this.busy = false;
0190     onComplete();
0191 }
0192
0193 playGoalDrop(onComplete: () => void): void {
0194     this.busy = true;
0195     const node = this.wholeNode;
0196     if (!node) {
0197         this.busy = false;
0198         onComplete();
0199         return;
0200     }
0201     const target = node.position.clone().add(new Vec3(0, -3, 0));
0202     tween(node)
0203         .delay(0.08)
0204         .to(0.48, { position: target }, { easing: 'quadIn' })
0205         .call(() => {
0206             node.active = false;
0207             this.busy = false;
0208             onComplete();
0209         })
0210         .start();
0211 }
0212
```

```
0213 private playWholeMove(result: MoveResult, onComplete: () => void, durationScale: number): void {
0214     const node = this.wholeNode;
0215     if (!node) {
0216         this.snapTo(result.current);
0217         onComplete();
0218         return;
0219     }
0220     this.busy = true;
0221     const start = result.previous.block;
0222     const startPosition = this.positionFor(start);
0223     const startRotation = this.rotationFor(start);
0224     const target = result.current.block;
0225     const targetPosition = this.positionFor(target);
0226     const targetRotation = this.rotationFor(target);
0227     const direction = this.directionVector(result.direction);
0228     const horizontalHalfExtent = this.horizontalHalfExtent(start, result.direction);
0229     const verticalHalfExtent = this.verticalHalfExtent(start);
0230     const pivot = startPosition.clone().add(new Vec3(
0231         direction.x * horizontalHalfExtent,
0232         -verticalHalfExtent,
0233         direction.z * horizontalHalfExtent,
0234     ));
0235     const relative = startPosition.clone().subtract(pivot);
0236     const axis = this.rotationAxis(result.direction);
0237     const fullyUnsupported = result.status === 'fallen'
0238         && (result.supportedCells?.length ?? 0) === 0;
0239     const progress = { value: 0 };
0240     this.motionTweenTarget = progress;
0241
0242     tween(progress)
0243         .to(this.rollDuration * durationScale, { value: 1 }, {
0244             easing: 'quadInOut',
0245             onUpdate: () => {
0246                 const rotationDelta = new Quat();
0247                 Quat.fromAxisAngle(rotationDelta, axis, Math.PI * 0.5 * progress.value);
0248                 const offset = new Vec3();
0249                 Vec3.transformQuat(offset, relative, rotationDelta);
0250                 node.setPosition(pivot.clone().add(offset));
0251                 const rotation = new Quat();
0252                 Quat.multiply(rotation, rotationDelta, startRotation);
0253                 node.setRotation(rotation);
0254             },
0255         })
0256         .call(() => {
0257             this.motionTweenTarget = null;
0258             if (fullyUnsupported) {
0259                 node.setPosition(targetPosition);
0260                 node.setRotation(targetRotation);
0261                 this.busy = false;
0262                 onComplete();
0263                 return;
0264             }
0265             node.setPosition(targetPosition);
```

```
0266         node.setRotation(targetRotation);
0267         this.snapTo(result.current);
0268         this.busy = false;
0269         onComplete();
0270     })
0271     .start();
0272 }
0273
0274 private playSplitMove(result: MoveResult, onComplete: () => void, durationScale: number): void {
0275     const active = result.previous.split?.activeCube ?? 0;
0276     const node = this.cubeNodes[active];
0277     if (!node) {
0278         this.snapTo(result.current);
0279         onComplete();
0280         return;
0281     }
0282     this.busy = true;
0283     const destination = result.current.split
0284         ? result.current.split.cubes[active]
0285         : result.occupiedCells.find((cell) =>
0286             cell.x !== result.previous.split?.cubes[1 - active].x
0287             || cell.z !== result.previous.split?.cubes[1 - active].z)
0288             ?? result.occupiedCells[0];
0289     tween(node)
0290         .to(0.16 * durationScale, { position: this.cubePosition(destination) }, { easing: 'quadInOut' })
0291         .call(() => {
0292             this.snapTo(result.current);
0293             this.busy = false;
0294             onComplete();
0295         })
0296         .start();
0297 }
0298
0299 private resolveNodes(): void {
0300     this.wholeNode ??= this.node.getChildByName('WholeBlock');
0301     this.cubeNodes = [
0302         this.cubeNodes[0] ?? this.node.getChildByName('SplitCubeA'),
0303         this.cubeNodes[1] ?? this.node.getChildByName('SplitCubeB'),
0304     ];
0305 }
0306
0307 private activeVisibleNode(): Node | null {
0308     const activeCube = this.cubeNodes[this.activeCubeIndex];
0309     return activeCube?.active ? activeCube : this.wholeNode;
0310 }
0311
0312 private cubePosition(coord: { x: number; z: number }): Vec3 {
0313     return new Vec3(coord.x * this.tileSize, 0.5, coord.z * this.tileSize);
0314 }
0315
0316 private positionFor(block: BlockState): Vec3 {
0317     const y = block.orientation === 'standing' ? 1 : 0.5;
0318     let x = block.anchor.x * this.tileSize;
```

```
0319     let z = block.anchor.z * this.tileSize;
0320     if (block.orientation === 'lying-x') x += this.tileSize * 0.5;
0321     else if (block.orientation === 'lying-z') z += this.tileSize * 0.5;
0322     return new Vec3(x, y, z);
0323 }
0324
0325 private rotationFor(block: BlockState): Quat {
0326     const rotation = new Quat();
0327     if (block.orientation === 'lying-x') Quat.fromEuler(rotation, 0, 0, 90);
0328     else if (block.orientation === 'lying-z') Quat.fromEuler(rotation, 90, 0, 0);
0329     return rotation;
0330 }
0331
0332 private directionVector(direction: Direction): Vec3 {
0333     if (direction === 'left') return new Vec3(-1, 0, 0);
0334     if (direction === 'right') return new Vec3(1, 0, 0);
0335     if (direction === 'up') return new Vec3(0, 0, -1);
0336     return new Vec3(0, 0, 1);
0337 }
0338
0339 private rotationAxis(direction: Direction): Vec3 {
0340     if (direction === 'left') return new Vec3(0, 0, 1);
0341     if (direction === 'right') return new Vec3(0, 0, -1);
0342     if (direction === 'up') return new Vec3(-1, 0, 0);
0343     return new Vec3(1, 0, 0);
0344 }
0345
0346 private horizontalHalfExtent(block: BlockState, direction: Direction): number {
0347     const movesAlongX = direction === 'left' || direction === 'right';
0348     if ((movesAlongX && block.orientation === 'lying-x')
0349         || (!movesAlongX && block.orientation === 'lying-z')) return this.tileSize;
0350     return this.tileSize * 0.5;
0351 }
0352
0353 private verticalHalfExtent(block: BlockState): number {
0354     return block.orientation === 'standing' ? this.tileSize : this.tileSize * 0.5;
0355 }
0356 }
0001 import {
0002     _decorator,
0003     Button,
0004     Component,
0005     EditBox,
0006     EventKeyboard,
0007     game,
0008     Game,
0009     input,
0010     Input,
0011     KeyCode,
0012     Label,
0013     Node,
0014     view,
0015 } from 'cc';
```

```
0016 import {
0017     calculateStarRating,
0018     createDefaultReleaseSave,
0019     getBestStarRating,
0020     getHighestUnlockedIndex,
0021     getLocalizedOnboardingCopy,
0022     languageDisplayName,
0023     nextGameLanguage,
0024     onboardingTopics,
0025     pendingOnboardingTopics,
0026     PuzzleEngine,
0027     resetCampaignProgress,
0028     resumeSavedRun,
0029     translate,
0030     withAcknowledgedTutorials,
0031     withUnlockedLevel,
0032     withBestStarRating,
0033 } from './shared/game/index';
0034 import type {
0035     Direction,
0036     MoveResult,
0037     OnboardingTopic,
0038     PuzzleAction,
0039     ReleaseSaveData,
0040     UiTextKey,
0041 } from './shared/game/index';
0042 import { chapterOneLevels, getLevelIndexByPasscode } from './shared/levels/index';
0043 import { createPlatformAdapter } from './shared/platform/index';
0044 import { AudioController } from './AudioController';
0045 import { BlockPresenter } from './BlockPresenter';
0046 import { BoardRenderer } from './BoardRenderer';
0047 import { CameraController } from './CameraController';
0048 import { ReleaseSaveRepository } from './ReleaseSaveRepository';
0049 import { OnboardingVisual } from './OnboardingVisual';
0050 import { TouchInputController } from './TouchInputController';
0051
0052 const { ccclass, property } = _decorator;
0053
0054 const oppositeDirection: Record<Direction, Direction> = {
0055     up: 'down',
0056     down: 'up',
0057     left: 'right',
0058     right: 'left',
0059 };
0060
0061 const keyboardDirections = new Map<KeyCode, Direction>([
0062     [KeyCode.ARROW_UP, 'down'],
0063     [KeyCode.KEY_W, 'down'],
0064     [KeyCode.ARROW_DOWN, 'up'],
0065     [KeyCode.KEY_S, 'up'],
0066     [KeyCode.ARROW_LEFT, 'right'],
0067     [KeyCode.KEY_A, 'right'],
0068     [KeyCode.ARROW_RIGHT, 'left'],
```

```
0069     [KeyCode.KEY_D, 'left'],
0070   });
0071
0072   type GameMode =
0073     | 'title'
0074     | 'stage-select'
0075     | 'passcode'
0076     | 'how-to-play'
0077     | 'credits'
0078     | 'playing'
0079     | 'paused'
0080     | 'tutorial'
0081     | 'failed'
0082     | 'completed'
0083     | 'campaign-complete';
0084
0085   @ccclass('GameplayController')
0086   export class GameplayController extends Component {
0087     @property(BoardRenderer)
0088     board: BoardRenderer | null = null;
0089
0090     @property(BlockPresenter)
0091     block: BlockPresenter | null = null;
0092
0093     @property(CameraController)
0094     cameraController: CameraController | null = null;
0095
0096     @property(TouchInputController)
0097     touchInput: TouchInputController | null = null;
0098
0099     @property(AudioController)
0100     audioController: AudioController | null = null;
0101
0102     @property(Label)
0103     levelStatLabel: Label | null = null;
0104
0105     @property(Label)
0106     movesStatLabel: Label | null = null;
0107
0108     @property(Label)
0109     timeStatLabel: Label | null = null;
0110
0111     @property(Label)
0112     passcodeStatLabel: Label | null = null;
0113
0114     @property(Node)
0115     gameplayHudRoot: Node | null = null;
0116
0117     @property(Node)
0118     pauseMenuRoot: Node | null = null;
0119
0120     @property(Node)
0121     titleMenuRoot: Node | null = null;
```

```
0122
0123 @property(Node)
0124 stageSelectRoot: Node | null = null;
0125
0126 @property(Node)
0127 passcodeRoot: Node | null = null;
0128
0129 @property(Node)
0130 howToPlayRoot: Node | null = null;
0131
0132 @property(Node)
0133 creditsRoot: Node | null = null;
0134
0135 @property(Node)
0136 newGameConfirmRoot: Node | null = null;
0137
0138 @property(Node)
0139 failureRoot: Node | null = null;
0140
0141 @property(Node)
0142 completionRoot: Node | null = null;
0143
0144 @property(Node)
0145 campaignCompleteRoot: Node | null = null;
0146
0147 @property(Node)
0148 tutorialRoot: Node | null = null;
0149
0150 @property(EditText)
0151 passcodeInput: EditText | null = null;
0152
0153 @property(Label)
0154 passcodeFeedback: Label | null = null;
0155
0156 @property(Label)
0157 soundToggleLabel: Label | null = null;
0158
0159 @property(Label)
0160 titleSoundLabel: Label | null = null;
0161
0162 @property(Label)
0163 titleLanguageLabel: Label | null = null;
0164
0165 @property(Label)
0166 pauseLanguageLabel: Label | null = null;
0167
0168 @property(Button)
0169 resumeButton: Button | null = null;
0170
0171 @property(Label)
0172 resumeButtonLabel: Label | null = null;
0173
0174 @property(Label)
```



```

0175     failureReasonLabel: Label | null = null;
0176
0177     @property(Label)
0178     completionStatsLabel: Label | null = null;
0179
0180     @property(Label)
0181     completionStarsLabel: Label | null = null;
0182
0183     @property(Label)
0184     completionContinueLabel: Label | null = null;
0185
0186     @property(Label)
0187     campaignStarsLabel: Label | null = null;
0188
0189     @property(Label)
0190     tutorialTitleLabel: Label | null = null;
0191
0192     @property(OnboardingVisual)
0193     tutorialVisual: OnboardingVisual | null = null;
0194
0195     @property(Label)
0196     tutorialBodyLabel: Label | null = null;
0197
0198     @property(Label)
0199     tutorialProgressLabel: Label | null = null;
0200
0201     @property(Label)
0202     tutorialNextLabel: Label | null = null;
0203
0204     @property(Label)
0205     howTopicTitleLabel: Label | null = null;
0206
0207     @property(OnboardingVisual)
0208     howTopicVisual: OnboardingVisual | null = null;
0209
0210     @property(Label)
0211     howTopicBodyLabel: Label | null = null;
0212
0213     @property(Label)
0214     howTopicProgressLabel: Label | null = null;
0215
0216     @property(Node)
0217     splitControlRoot: Node | null = null;
0218
0219     stageButtons: Button[] = [];
0220     stageButtonLabels: Label[] = [];
0221     stageStarLabels: Label[] = [];
0222     completedPasscodeButtons: Button[] = [];
0223     completedPasscodeLevelLabels: Label[] = [];
0224     completedPasscodeValueLabels: Label[] = [];
0225     localizedLabels: Partial<Record<UiTextKey, Label[]>> = {};
0226
0227     private readonly platform = createPlatformAdapter();

```

```
0228     private readonly saveRepository = new ReleaseSaveRepository(chapterOneLevels);
0229     private mode: GameMode = 'title';
0230     private levelIndex = 0;
0231     private engine: PuzzleEngine | null = null;
0232     private bufferedMove: Direction | null = null;
0233     private elapsedSeconds = 0;
0234     private displayedSecond = -1;
0235     private actionLog: PuzzleAction[] = [];
0236     private saveData: ReleaseSaveData = createDefaultReleaseSave(chapterOneLevels);
0237     private tutorialQueue: OnboardingTopic[] = [];
0238     private tutorialTotal = 0;
0239     private howTopicIndex = 0;
0240     private titleAttractEngine: PuzzleEngine | null = null;
0241     private titleAttractRoute: readonly Direction[] = [];
0242     private titleAttractStep = 0;
0243
0244     start(): void {
0245         if (this.touchInput) {
0246             this.touchInput.onMove = (direction) => this.requestMove(direction);
0247         }
0248         this.saveData = this.saveRepository.load();
0249         this.audioController?.setSoundEnabled(this.saveData.soundEnabled);
0250         this.applyLocalization();
0251         game.on(Game.EVENT_HIDE, this.handleGameHide, this);
0252         input.on(Input.EventType.KEY_DOWN, this.handleKeyDown, this);
0253         this.showTitleMenu();
0254     }
0255
0256     onDestroy(): void {
0257         this.stopTitleAttract();
0258         game.off(Game.EVENT_HIDE, this.handleGameHide, this);
0259         input.off(Input.EventType.KEY_DOWN, this.handleKeyDown, this);
0260     }
0261
0262     update(deltaTime: number): void {
0263         const state = this.engine?.getState();
0264         if (this.mode !== 'playing' || !state || state.completed || state.failed) return;
0265         this.elapsedSeconds += deltaTime;
0266         const second = Math.floor(this.elapsedSeconds);
0267         if (second !== this.displayedSecond) {
0268             this.displayedSecond = second;
0269             this.updateHud();
0270         }
0271     }
0272
0273     startGame(): void {
0274         this.audioController?.playUi();
0275         if (this.hasCampaignProgress()) {
0276             this.showOnly(this.newGameConfirmRoot);
0277             return;
0278         }
0279         this.confirmNewGame();
0280     }
```

```
0281
0282   confirmNewGame(): void {
0283       this.audioController?.playUi();
0284       this.saveData = resetCampaignProgress(this.saveData, chapterOneLevels);
0285       this.saveRepository.save(this.saveData);
0286       this.loadLevel(0);
0287   }
0288
0289   cancelNewGame(): void {
0290       this.audioController?.playUi();
0291       this.showTitleMenu();
0292   }
0293
0294   resumeGame(): void {
0295       const resumed = resumeSavedRun(this.saveData, chapterOneLevels);
0296       if (!resumed) {
0297           this.updateTitleMenu();
0298           return;
0299       }
0300       this.audioController?.playUi();
0301       this.presentRun(resumed.levelIndex, resumed.engine, resumed.actions, resumed.elapsedSeconds);
0302   }
0303
0304   openStageSelect(): void {
0305       this.audioController?.playUi();
0306       this.mode = 'stage-select';
0307       this.bufferedMove = null;
0308       this.refreshStageButtons();
0309       this.showOnly(this.stageSelectRoot);
0310   }
0311
0312   openPasscode(): void {
0313       this.audioController?.playUi();
0314       this.mode = 'passcode';
0315       this.bufferedMove = null;
0316       if (this.passcodeInput) this.passcodeInput.string = '';
0317       if (this.passcodeFeedback) this.passcodeFeedback.string = '';
0318       this.refreshCompletedPasscodes();
0319       this.showOnly(this.passcodeRoot);
0320   }
0321
0322   selectStage(_event: unknown, customEventData: string): void {
0323       const index = Number.parseInt(customEventData, 10);
0324       if (!Number.isInteger(index)
0325           || index < 0
0326           || index > getHighestUnlockedIndex(this.saveData, chapterOneLevels)) return;
0327       this.audioController?.playUi();
0328       this.loadLevel(index);
0329   }
0330
0331   submitPasscode(): void {
0332       const index = getLevelIndexByPasscode(this.passcodeInput?.string ?? '');
0333       if (index < 0) {
```

```
0334         if (this.passcodeFeedback) {
0335             this.passcodeFeedback.string = translate(this.saveData.language, 'invalidPasscode');
0336         }
0337         return;
0338     }
0339     this.audioController?.playUi();
0340     this.loadLevel(index);
0341 }
0342
0343 selectCompletedPasscode(_event: unknown, customEventData: string): void {
0344     const index = Number.parseInt(customEventData, 10);
0345     if (!Number.isInteger(index))
0346         || index < 0
0347         || index >= chapterOneLevels.length
0348         || getBestStarRating(this.saveData, chapterOneLevels[index].id) === 0) return;
0349     this.audioController?.playUi();
0350     this.loadLevel(index);
0351 }
0352
0353 showHowToPlay(): void {
0354     this.audioController?.playUi();
0355     this.mode = 'how-to-play';
0356     this.bufferedMove = null;
0357     this.howTopicIndex = 0;
0358     this.updateHowToPlay();
0359     this.showOnly(this.howToPlayRoot);
0360 }
0361
0362 previousHowTopic(): void {
0363     if (this.mode !== 'how-to-play') return;
0364     this.audioController?.playUi();
0365     this.howTopicIndex = (this.howTopicIndex - 1 + onboardingTopics.length) % onboardingTopics.length;
0366     this.updateHowToPlay();
0367 }
0368
0369 nextHowTopic(): void {
0370     if (this.mode !== 'how-to-play') return;
0371     this.audioController?.playUi();
0372     this.howTopicIndex = (this.howTopicIndex + 1) % onboardingTopics.length;
0373     this.updateHowToPlay();
0374 }
0375
0376 acknowledgeTutorial(): void {
0377     if (this.mode !== 'tutorial' || this.tutorialQueue.length === 0) return;
0378     this.audioController?.playUi();
0379     this.acknowledgeTopics([this.tutorialQueue[0]]);
0380     this.tutorialQueue.shift();
0381     this.advanceTutorial();
0382 }
0383
0384 skipTutorial(): void {
0385     if (this.mode !== 'tutorial') return;
0386     this.audioController?.playUi();
```

```
0387     this.acknowledgeTopics(this.tutorialQueue);
0388     this.tutorialQueue = [];
0389     this.advanceTutorial();
0390 }
0391
0392 showCredits(): void {
0393     this.audioController?.playUi();
0394     this.mode = 'credits';
0395     this.bufferedMove = null;
0396     this.showOnly(this.creditsRoot);
0397 }
0398
0399 returnToTitle(): void {
0400     this.audioController?.playUi();
0401     this.showTitleMenu();
0402 }
0403
0404 openMenu(): void {
0405     if (this.mode !== 'playing' || this.block?.isBusy()) return;
0406     this.audioController?.playUi();
0407     this.persistCurrentRun();
0408     this.mode = 'paused';
0409     this.bufferedMove = null;
0410     if (this.pauseMenuRoot) this.pauseMenuRoot.active = true;
0411     this.updateSoundLabels();
0412 }
0413
0414 returnToGame(): void {
0415     if (this.mode !== 'paused') return;
0416     this.audioController?.playUi();
0417     this.mode = 'playing';
0418     if (this.pauseMenuRoot) this.pauseMenuRoot.active = false;
0419     this.updateHud();
0420 }
0421
0422 toggleSound(): void {
0423     const enabled = this.audioController?.toggleSound() ?? true;
0424     this.saveData = { ...this.saveData, soundEnabled: enabled };
0425     this.saveRepository.save(this.saveData);
0426     if (enabled) this.audioController?.playUi();
0427     this.updateSoundLabels();
0428 }
0429
0430 cycleLanguage(): void {
0431     this.audioController?.playUi();
0432     this.saveData = {
0433         ...this.saveData,
0434         language: nextGameLanguage(this.saveData.language),
0435     };
0436     this.saveRepository.save(this.saveData);
0437     this.applyLocalization();
0438 }
0439
```

```
0440     switchCube(): void {
0441         if (this.mode !== 'playing' || !this.engine?.getState().split || this.block?.isBusy()) return;
0442         this.audioController?.playUi();
0443         const state = this.engine.switchActiveCube();
0444         this.actionLog.push('switch-cube');
0445         this.block?.snapTo(state);
0446         this.persistCurrentRun();
0447         this.updateHud();
0448     }
0449
0450     quitToMenu(): void {
0451         if (this.mode !== 'paused') return;
0452         this.audioController?.playUi();
0453         this.persistCurrentRun();
0454         this.showTitleMenu();
0455     }
0456
0457     retryLevel(): void {
0458         if (this.mode !== 'failed') return;
0459         this.audioController?.playUi();
0460         this.loadLevel(this.levelIndex);
0461     }
0462
0463     replayLevel(): void {
0464         if (this.mode !== 'completed') return;
0465         this.audioController?.playUi();
0466         this.loadLevel(this.levelIndex);
0467     }
0468
0469     continueAfterComplete(): void {
0470         if (this.mode !== 'completed') return;
0471         this.audioController?.playUi();
0472         if (this.levelIndex + 1 < chapterOneLevels.length) this.loadLevel(this.levelIndex + 1);
0473         else this.showTitleMenu();
0474     }
0475
0476     quitResultToMenu(): void {
0477         if (this.mode !== 'failed' && this.mode !== 'completed') return;
0478         this.audioController?.playUi();
0479         this.showTitleMenu();
0480     }
0481
0482     requestMove(direction: Direction): void {
0483         if (this.mode !== 'playing' || !this.engine || !this.block) return;
0484         const state = this.engine.getState();
0485         if (state.completed || state.failed) return;
0486         if (this.block.isBusy()) {
0487             this.bufferedMove = direction;
0488             return;
0489         }
0490
0491         this.audioController?.beginInteraction();
0492         const result = this.engine.move(direction);
```

```
0493     this.presentMove(result);
0494 }
0495
0496 private loadLevel(index: number): void {
0497     this.presentRun(index, new PuzzleEngine(chapterOneLevels[index]), [], 0);
0498 }
0499
0500 private presentRun(
0501     index: number,
0502     engine: PuzzleEngine,
0503     actions: readonly PuzzleAction[],
0504     elapsedSeconds: number,
0505 ): void {
0506     this.stopTitleAttract();
0507     this.bufferedMove = null;
0508     this.levelIndex = index;
0509     this.engine = engine;
0510     this.actionLog = [...actions];
0511     this.elapsedSeconds = elapsedSeconds;
0512     this.displayedSecond = Math.floor(elapsedSeconds);
0513     this.mode = 'playing';
0514     this.hideAllOverlays();
0515     if (this.gameplayHudRoot) this.gameplayHudRoot.active = true;
0516
0517     const level = chapterOneLevels[index];
0518     this.board?.render(level);
0519     const state = engine.getState();
0520     this.board?.applyState(state);
0521     this.block?.snapTo(state);
0522     this.cameraController?.frameLevel(level);
0523     this.platform.onLevelStarted(level.id);
0524     this.persistCurrentRun();
0525     this.updateHud();
0526     this.startAutomaticTutorial(level);
0527 }
0528
0529 private showTitleMenu(): void {
0530     this.stopTitleAttract();
0531     this.mode = 'title';
0532     this.bufferedMove = null;
0533     this.engine = null;
0534     this.actionLog = [];
0535     if (this.gameplayHudRoot) this.gameplayHudRoot.active = false;
0536     this.showOnly(this.titleMenuRoot);
0537     if (this.passcodeInput) this.passcodeInput.string = '';
0538     if (this.passcodeFeedback) this.passcodeFeedback.string = '';
0539
0540     const attractIndex = Math.min(2, chapterOneLevels.length - 1);
0541     const level = chapterOneLevels[attractIndex];
0542     const attractEngine = new PuzzleEngine(level);
0543     const forwardRoute = (level.solution ?? [])
0544         .filter((action): action is Direction => action !== 'switch-cube')
0545         .slice(0, -1);
```

```
0546     this.titleAttractEngine = attractEngine;
0547     this.titleAttractRoute = [
0548         ...forwardRoute,
0549         ...[...forwardRoute].reverse().map((direction) => oppositeDirection[direction]),
0550     ];
0551     this.titleAttractStep = 0;
0552     this.board?.render(level);
0553     this.board?.applyState(attractEngine.getState());
0554     this.block?.startAttract(attractEngine.getState());
0555     const titleScreenOffset = Math.min(0.45, 390 / Math.max(1, view.getVisibleSize().width));
0556     this.cameraController?.frameLevel(level, titleScreenOffset);
0557     this.scheduleOnce(this.advanceTitleAttract, 0.55);
0558     this.updateTitleMenu();
0559 }
0560
0561 private advanceTitleAttract(): void {
0562     const engine = this.titleAttractEngine;
0563     const block = this.block;
0564     const direction = this.titleAttractRoute[this.titleAttractStep];
0565     if (this.mode !== 'title' || !engine || !block || !direction) return;
0566
0567     const result = engine.move(direction);
0568     block.playMove(result, () => {
0569         if (this.mode !== 'title' || this.titleAttractEngine !== engine) return;
0570         this.board?.applyState(result.current);
0571         this.titleAttractStep = (this.titleAttractStep + 1) % this.titleAttractRoute.length;
0572         this.scheduleOnce(this.advanceTitleAttract, 0.24);
0573     }, 2);
0574 }
0575
0576 private stopTitleAttract(): void {
0577     this.unschedule(this.advanceTitleAttract);
0578     this.titleAttractEngine = null;
0579     this.titleAttractRoute = [];
0580     this.titleAttractStep = 0;
0581     this.block?.stopAttract();
0582 }
0583
0584 private presentMove(result: MoveResult): void {
0585     if (!this.block || !this.engine) return;
0586     this.block.playMove(result, () => {
0587         this.board?.applyState(result.current);
0588         if (result.status === 'fallen') {
0589             const brokeFragileTile = result.message?.includes('fragile') === true;
0590             if (brokeFragileTile && result.occupiedCells[0]) {
0591                 this.board?.playFragileBreak(result.occupiedCells[0]);
0592                 this.audioController?.playGlassBreak();
0593             }
0594             this.audioController?.playFall();
0595             this.platform.vibrateLight();
0596             this.updateHud();
0597             this.block?.playFall(result, () => this.showFailure(result));
0598             return;
0599         }
0600     });
0601 }
```



```
0599     }
0600
0601     this.audioController?.playMove();
0602     this.actionLog.push(result.direction);
0603     this.updateHud();
0604     if (result.status === 'completed') {
0605         this.completeLevel(result);
0606         return;
0607     }
0608
0609     this.persistCurrentRun();
0610     const buffered = this.bufferedMove;
0611     this.bufferedMove = null;
0612     if (buffered) this.requestMove(buffered);
0613   });
0614 }
0615
0616 private showFailure(result: MoveResult): void {
0617   this.mode = 'failed';
0618   this.bufferedMove = null;
0619   if (this.failureReasonLabel) {
0620     this.failureReasonLabel.string = result.message?.includes('fragile')
0621       ? translate(this.saveData.language, 'failureFragile')
0622       : translate(this.saveData.language, 'failureVoid');
0623   }
0624   this.hideAllOverlays();
0625   if (this.failureRoot) this.failureRoot.active = true;
0626 }
0627
0628 private completeLevel(result: MoveResult): void {
0629   this.audioController?.playComplete();
0630   this.platform.onLevelCompleted(result.current.levelId, result.current.steps);
0631   this.platform.vibrateLight();
0632   const level = chapterOneLevels[this.levelIndex];
0633   const optimalMoves = level.par
0634     ?? level.solution?.filter((action) => action !== 'switch-cube').length
0635     ?? result.current.steps;
0636   const rating = calculateStarRating(optimalMoves, result.current.steps);
0637   const nextIndex = Math.min(this.levelIndex + 1, chapterOneLevels.length - 1);
0638   this.saveData = withUnlockedLevel(this.saveData, nextIndex, chapterOneLevels);
0639   this.saveData = withBestStarRating(this.saveData, level.id, rating);
0640   this.saveData = {
0641     ...this.saveData,
0642     currentRun: this.levelIndex + 1 < chapterOneLevels.length
0643       ? { levelId: chapterOneLevels[this.levelIndex + 1].id, actions: [], elapsedSeconds: 0 }
0644       : null,
0645   };
0646   this.saveRepository.save(this.saveData);
0647   if (this.completionStatsLabel) {
0648     this.completionStatsLabel.string = translate(this.saveData.language, 'completionStats', {
0649       moves: result.current.steps,
0650       optimal: optimalMoves,
0651       time: this.formattedTime(),
```

```
0652         passcode: level.passcode,
0653     });
0654 }
0655 if (this.completionStarsLabel) {
0656     this.completionStarsLabel.string = this.starText(rating);
0657 }
0658 if (this.completionContinueLabel) {
0659     this.completionContinueLabel.string = this.levelIndex + 1 < chapterOneLevels.length
0660         ? translate(this.saveData.language, 'continue')
0661         : translate(this.saveData.language, 'returnToMenu');
0662 }
0663 const completedAllLevels = chapterOneLevels.every(
0664     (candidate) => getBestStarRating(this.saveData, candidate.id) > 0,
0665 );
0666 if (completedAllLevels && this.campaignStarsLabel) {
0667     const earnedStars = chapterOneLevels.reduce(
0668         (total, candidate) => total + getBestStarRating(this.saveData, candidate.id),
0669         0,
0670     );
0671     this.campaignStarsLabel.string = translate(this.saveData.language, 'campaignStars', {
0672         stars: earnedStars,
0673         maxStars: chapterOneLevels.length * 3,
0674     });
0675 }
0676 this.block?.playGoalDrop() => {
0677     this.mode = completedAllLevels ? 'campaign-complete' : 'completed';
0678     this.bufferedMove = null;
0679     this.hideAllOverlays();
0680     const resultRoot = completedAllLevels ? this.campaignCompleteRoot : this.completionRoot;
0681     if (resultRoot) resultRoot.active = true;
0682 };
0683 }
0684
0685 private handleGameHide(): void {
0686     this.persistCurrentRun();
0687     if (this.mode !== 'playing') return;
0688     this.mode = 'paused';
0689     this.bufferedMove = null;
0690     if (this.pauseMenuRoot) this.pauseMenuRoot.active = true;
0691     this.updateSoundLabels();
0692 }
0693
0694 private handleKeyDown(event: EventKeyboard): void {
0695     if (this.mode === 'playing') {
0696         const direction = keyboardDirections.get(event.keyCode);
0697         if (direction) {
0698             this.requestMove(direction);
0699             return;
0700         }
0701         if (event.keyCode === KeyCode.SPACE) {
0702             this.switchCube();
0703             return;
0704         }
0705     }
```

```
0705     }
0706     if (event.keyCode !== KeyCode.MOBILE_BACK && event.keyCode !== KeyCode.ESCAPE) return;
0707     if (this.mode === 'playing') {
0708         this.openMenu();
0709         return;
0710     }
0711     if (this.mode === 'paused') {
0712         this.returnToGame();
0713         return;
0714     }
0715     if (this.mode === 'tutorial') {
0716         this.skipTutorial();
0717         return;
0718     }
0719     if (this.mode !== 'title') this.showTitleMenu();
0720 }
0721
0722 private persistCurrentRun(): void {
0723     const state = this.engine?.getState();
0724     if (!state || state.failed || state.completed) return;
0725     this.saveData = {
0726         ...this.saveData,
0727         currentRun: {
0728             levelId: chapterOneLevels[this.levelIndex].id,
0729             actions: [...this.actionLog],
0730             elapsedSeconds: this.elapsedSeconds,
0731         },
0732     };
0733     this.saveRepository.save(this.saveData);
0734 }
0735
0736 private startAutomaticTutorial(level: typeof chapterOneLevels[number]): void {
0737     this.tutorialQueue = pendingOnboardingTopics(level, this.saveData.acknowledgedTutorials);
0738     this.tutorialTotal = this.tutorialQueue.length;
0739     if (this.tutorialQueue.length === 0) return;
0740     this.mode = 'tutorial';
0741     this.bufferedMove = null;
0742     this.hideAllOverlays();
0743     if (this.tutorialRoot) this.tutorialRoot.active = true;
0744     this.updateTutorial();
0745 }
0746
0747 private advanceTutorial(): void {
0748     if (this.tutorialQueue.length > 0) {
0749         this.updateTutorial();
0750         return;
0751     }
0752     this.mode = 'playing';
0753     if (this.tutorialRoot) this.tutorialRoot.active = false;
0754     this.updateHud();
0755 }
0756
0757 private acknowledgeTopics(topics: readonly OnboardingTopic[]): void {
```

```
0758     this.saveData = withAcknowledgedTutorials(  
0759         this.saveData,  
0760         topics.map((topic) => topic.id),  
0761     );  
0762     this.saveRepository.save(this.saveData);  
0763 }  
0764  
0765 private updateTutorial(): void {  
0766     const topic = this.tutorialQueue[0];  
0767     if (!topic) return;  
0768     const copy = getLocalizedOnboardingCopy(this.saveData.language, topic.id);  
0769     if (this.tutorialTitleLabel) this.tutorialTitleLabel.string = copy.title;  
0770     this.tutorialVisual?.setTopic(topic.id);  
0771     if (this.tutorialBodyLabel) this.tutorialBodyLabel.string = copy.body;  
0772     if (this.tutorialProgressLabel) {  
0773         const position = this.tutorialTotal - this.tutorialQueue.length + 1;  
0774         this.tutorialProgressLabel.string = `${position} / ${this.tutorialTotal}`;  
0775     }  
0776     if (this.tutorialNextLabel) {  
0777         this.tutorialNextLabel.string = this.tutorialQueue.length === 1  
0778             ? translate(this.saveData.language, 'gotIt')  
0779             : translate(this.saveData.language, 'next');  
0780     }  
0781 }  
0782  
0783 private updateHowToPlay(): void {  
0784     const topic = onboardingTopics[this.howTopicIndex];  
0785     if (!topic) return;  
0786     const copy = getLocalizedOnboardingCopy(this.saveData.language, topic.id);  
0787     if (this.howTopicTitleLabel) this.howTopicTitleLabel.string = copy.title;  
0788     this.howTopicVisual?.setTopic(topic.id);  
0789     if (this.howTopicBodyLabel) this.howTopicBodyLabel.string = copy.body;  
0790     if (this.howTopicProgressLabel) {  
0791         this.howTopicProgressLabel.string = `${this.howTopicIndex + 1} / ${onboardingTopics.length}`;  
0792     }  
0793 }  
0794  
0795 private refreshStageButtons(): void {  
0796     const highest = getHighestUnlockedIndex(this.saveData, chapterOneLevels);  
0797     this.stageButtons.forEach((button, index) => {  
0798         const unlocked = index <= highest;  
0799         button.interactable = unlocked;  
0800         const label = this.stageButtonLabels[index];  
0801         if (label) {  
0802             label.string = unlocked ? this.twoDigits(index + 1) : '---';  
0803         }  
0804         const stars = this.stageStarLabels[index];  
0805         if (stars) {  
0806             const rating = unlocked  
0807                 ? getBestStarRating(this.saveData, chapterOneLevels[index].id)  
0808                 : 0;  
0809             stars.string = rating > 0 ? this.starText(rating) : '';  
0810         }  
0811     })  
}
```

```
0811     });
0812 }
0813
0814 private refreshCompletedPasscodes(): void {
0815     this.completedPasscodeButtons.forEach((button, levelIndex) => {
0816         const level = chapterOneLevels[levelIndex];
0817         const completed = level !== undefined
0818             && getBestStarRating(this.saveData, level.id) > 0;
0819         button.node.active = level !== undefined;
0820         button.interactable = completed;
0821         if (!level) return;
0822         const column = levelIndex % 6;
0823         const row = Math.floor(levelIndex / 6);
0824         button.node.setPosition((column - 2.5) * 116, 22 - row * 38, 0);
0825         const levelLabel = this.completedPasscodeLevelLabels[levelIndex];
0826         if (levelLabel) levelLabel.string = this.twoDigits(levelIndex + 1);
0827         const valueLabel = this.completedPasscodeValueLabels[levelIndex];
0828         if (valueLabel) valueLabel.string = completed ? level.passcode : '*****';
0829     });
0830 }
0831
0832 private updateTitleMenu(): void {
0833     const resumed = resumeSavedRun(this.saveData, chapterOneLevels);
0834     if (this.resumeButton) this.resumeButton.interactable = resumed !== null;
0835     if (this.resumeButtonLabel) {
0836         this.resumeButtonLabel.string = resumed
0837             ? translate(this.saveData.language, 'resumeGameStage', {
0838                 stage: this.twoDigits(resumed.levelIndex + 1),
0839             })
0840             : translate(this.saveData.language, 'resumeGame');
0841     }
0842     this.updateSoundLabels();
0843 }
0844
0845 private updateHud(): void {
0846     const state = this.engine?.getState();
0847     if (!state) return;
0848     const level = chapterOneLevels[this.levelIndex];
0849     if (this.levelStatLabel) {
0850         this.levelStatLabel.string = `${translate(this.saveData.language, 'level')}\n${this.twoDigits(this.levelIndex + 1)} /
0851         ${chapterOneLevels.length}`;
0852     }
0853     if (this.movesStatLabel) {
0854         this.movesStatLabel.string = `${translate(this.saveData.language, 'moves')}\n${state.steps}`;
0855     }
0856     if (this.timeStatLabel) {
0857         this.timeStatLabel.string = `${translate(this.saveData.language, 'time')}\n${this.formattedTime()}`;
0858     }
0859     if (this.passcodeStatLabel) {
0860         this.passcodeStatLabel.string = `${translate(this.saveData.language, 'passcode')}\n${level.passcode}`;
0861     }
0862     if (this.splitControlRoot) {
0863         this.splitControlRoot.active = this.mode === 'playing' && state.split !== null;
0864     }
0865 }
```

```
0864     }
0865
0866     private updateSoundLabels(): void {
0867         const state = translate(
0868             this.saveData.language,
0869             this.audioController?.isSoundEnabled() === false ? 'soundOff' : 'soundOn',
0870         );
0871         const text = translate(this.saveData.language, 'soundSetting', { state });
0872         if (this.soundToggleLabel) this.soundToggleLabel.string = text;
0873         if (this.titleSoundLabel) this.titleSoundLabel.string = text;
0874     }
0875
0876     private applyLocalization(): void {
0877         const language = this.saveData.language;
0878         for (const key of Object.keys(this.localizedLabels) as UiTextKey[]) {
0879             for (const label of this.localizedLabels[key] ?? []) {
0880                 label.string = translate(language, key);
0881             }
0882         }
0883
0884         const passcodeText = translate(language, 'passcode');
0885         if (this.passcodeInput) this.passcodeInput.placeholder = passcodeText;
0886         if (this.passcodeFeedback?.string) {
0887             this.passcodeFeedback.string = translate(language, 'invalidPasscode');
0888         }
0889
0890         const languageText = translate(language, 'languageSetting', {
0891             language: languageDisplayName(language),
0892         });
0893         if (this.titleLanguageLabel) this.titleLanguageLabel.string = languageText;
0894         if (this.pauseLanguageLabel) this.pauseLanguageLabel.string = languageText;
0895
0896         this.updateTitleMenu();
0897         this.updateHud();
0898         if (this.mode === 'tutorial') this.updateTutorial();
0899         if (this.mode === 'how-to-play') this.updateHowToPlay();
0900     }
0901
0902     private showOnly(root: Node | null): void {
0903         this.hideAllOverlays();
0904         if (root) root.active = true;
0905     }
0906
0907     private hideAllOverlays(): void {
0908         for (const root of [
0909             this.titleMenuRoot,
0910             this.pauseMenuRoot,
0911             this.stageSelectRoot,
0912             this.passcodeRoot,
0913             this.howToPlayRoot,
0914             this.creditsRoot,
0915             this.newGameConfirmRoot,
0916             this.failureRoot,
```

```

0917         this.completionRoot,
0918         this.campaignCompleteRoot,
0919         this.tutorialRoot,
0920     }) {
0921         if (root) root.active = false;
0922     }
0923 }
0924
0925 private hasCampaignProgress(): boolean {
0926     return this.saveData.currentRun !== null
0927         || getHighestUnlockedIndex(this.saveData, chapterOneLevels) > 0;
0928 }
0929
0930 private formattedTime(): string {
0931     const minutes = Math.floor(this.elapsedSeconds / 60);
0932     const seconds = Math.floor(this.elapsedSeconds % 60);
0933     return `${this.twoDigits(minutes)}:${this.twoDigits(seconds)}`;
0934 }
0935
0936 private twoDigits(value: number): string {
0937     return value < 10 ? `0${value}` : String(value);
0938 }
0939
0940 private starText(rating: number): string {
0941     return ['☆☆', '☆☆', '☆☆', '☆☆'][rating] ?? '☆☆';
0942 }
0943 }
0001 import { _decorator, Color, Component, Graphics } from 'cc';
0002 import type { TutorialTopicId } from './shared/game/index';
0003
0004 const { ccclass } = _decorator;
0005 type Point = readonly [number, number];
0006 type BoardPoint = readonly [number, number, number];
0007
0008 // Orthographic screen basis produced by the gameplay camera offset (-6, 13, -18).
0009 const BOARD_SCREEN_X: Point = [-0.9486832981, 0.1787374330];
0010 const BOARD_SCREEN_Y: Point = [0, 0.8249419983];
0011 const BOARD_SCREEN_Z: Point = [0.3162277660, 0.5362122989];
0012 const BOARD_PROJECTION_SCALE = 58;
0013
0014 const COLORS = {
0015     stoneTop: new Color(238, 239, 242, 255),
0016     stoneSide: new Color(157, 162, 172, 255),
0017     stoneEdge: new Color(112, 118, 128, 255),
0018     glassTop: new Color(184, 226, 234, 112),
0019     glassSide: new Color(112, 172, 187, 92),
0020     glassEdge: new Color(218, 244, 248, 205),
0021     softSwitch: new Color(39, 127, 114, 255),
0022     hardSwitch: new Color(173, 94, 35, 255),
0023     splitSwitch: new Color(102, 91, 158, 255),
0024     bridgeTop: new Color(78, 151, 159, 255),
0025     bridgeSide: new Color(40, 80, 91, 255),
0026     blockTop: new Color(218, 48, 57, 255),

```

```

0027     blockLeft: new Color(166, 18, 29, 255),
0028     blockRight: new Color(118, 12, 22, 255),
0029     blockEdge: new Color(190, 190, 190, 255),
0030     goal: new Color(4, 5, 8, 255),
0031     goalRim: new Color(105, 108, 116, 255),
0032     guide: new Color(218, 185, 105, 255),
0033 };
0034
0035 @ccclass('OnboardingVisual')
0036 export class OnboardingVisual extends Component {
0037     private graphics: Graphics | null = null;
0038     private topic: TutorialTopicId | null = null;
0039
0040     setTopic(topic: TutorialTopicId): void {
0041         this.topic = topic;
0042         this.drawTopic(topic);
0043     }
0044
0045     onEnable(): void {
0046         if (this.topic) this.drawTopic(this.topic);
0047     }
0048
0049     private drawTopic(topic: TutorialTopicId): void {
0050         const graphics = this.graphics ?? this.node.getComponent(Graphics) ?? this.node.addComponent(Graphics);
0051         this.graphics = graphics;
0052         graphics.clear();
0053         switch (topic) {
0054             case 'movement': this.drawMovement(graphics); break;
0055             case 'goal': this.drawGoal(graphics); break;
0056             case 'fragile': this.drawFragile(graphics); break;
0057             case 'soft-switch': this.drawSoftSwitch(graphics); break;
0058             case 'hard-switch': this.drawHardSwitch(graphics); break;
0059             case 'bridge': this.drawBridge(graphics); break;
0060             case 'split': this.drawSplit(graphics); break;
0061             case 'switch-cube': this.drawSwitchCube(graphics); break;
0062             case 'recombine': this.drawRecombine(graphics); break;
0063         }
0064     }
0065
0066     private drawMovement(graphics: Graphics): void {
0067         this.drawTile(graphics, 0, -7, 'stone', 58, 30);
0068         this.drawBlock(graphics, 0, 2, 20, 28);
0069         this.drawChevron(graphics, 0, 43, 0, 7);
0070         this.drawChevron(graphics, 0, -40, 0, -7);
0071         this.drawChevron(graphics, -43, 1, -7, 0);
0072         this.drawChevron(graphics, 43, 1, 7, 0);
0073     }
0074
0075     private drawGoal(graphics: Graphics): void {
0076         this.drawTile(graphics, -27, -11, 'stone', 43, 22);
0077         this.drawTile(graphics, 27, -11, 'stone', 43, 22);
0078         this.drawHole(graphics, 0, 12, 48, 25);
0079         this.drawBlock(graphics, 0, 21, 17, 23);

```



```
0080     }
0081
0082     private drawFragile(graphics: Graphics): void {
0083         this.drawTile(graphics, 0, 2, 'glass', 76, 40);
0084         this.strokePolygon(graphics, [[-31, 2], [0, 18], [31, 2], [0, -14]], COLORS.glassEdge, 1.5);
0085     }
0086
0087     private drawSoftSwitch(graphics: Graphics): void {
0088         this.drawTile(graphics, 0, 0, 'stone', 76, 40);
0089         graphics.fillColor = COLORS.softSwitch;
0090         graphics.ellipse(0, 2, 18, 9);
0091         graphics.fill();
0092         graphics.fillColor = COLORS.stoneTop;
0093         graphics.ellipse(0, 4, 11, 5.5);
0094         graphics.fill();
0095     }
0096
0097     private drawHardSwitch(graphics: Graphics): void {
0098         this.drawProjectedStoneTile(graphics);
0099         this.fillPolygon(graphics, this.projectedBoxTop(0.58, 0.1, 0.1825, 45), COLORS.hardSwitch);
0100         this.fillPolygon(graphics, this.projectedBoxTop(0.58, 0.1, 0.1845, -45), COLORS.hardSwitch);
0101     }
0102
0103     private drawBridge(graphics: Graphics): void {
0104         this.drawTile(graphics, -22, 12, 'bridge', 48, 25);
0105         this.drawTile(graphics, 22, -12, 'bridge', 48, 25);
0106         graphics.strokeColor = COLORS.guide;
0107         graphics.lineWidth = 2;
0108         graphics.moveTo(-34, -28);
0109         graphics.lineTo(-26, -34);
0110         graphics.lineTo(-18, -28);
0111         graphics.moveTo(18, 32);
0112         graphics.lineTo(26, 38);
0113         graphics.lineTo(34, 32);
0114         graphics.stroke();
0115     }
0116
0117     private drawSplit(graphics: Graphics): void {
0118         this.drawProjectedStoneTile(graphics);
0119         this.fillPolygon(graphics, this.projectedTriangle(-0.22, 0, 0.21, 0.175, -90), COLORS.splitSwitch);
0120         this.fillPolygon(graphics, this.projectedTriangle(0.22, 0, 0.21, 0.175, 90), COLORS.splitSwitch);
0121         this.fillPolygon(graphics, this.projectedBoxTop(0.045, 0.5, 0.175, 0), COLORS.splitSwitch);
0122     }
0123
0124     private drawSwitchCube(graphics: Graphics): void {
0125         this.drawTile(graphics, -23, -9, 'stone', 45, 23);
0126         this.drawTile(graphics, 23, 9, 'stone', 45, 23);
0127         this.drawBlock(graphics, -23, -1, 17, 18);
0128         this.drawBlock(graphics, 23, 17, 17, 18);
0129         graphics.strokeColor = COLORS.guide;
0130         graphics.lineWidth = 2;
0131         graphics.ellipse(-23, 0, 20, 11);
0132         graphics.stroke();
```

```
0133     }
0134
0135     private drawRecombine(graphics: Graphics): void {
0136         this.drawTile(graphics, -24, -7, 'stone', 46, 24);
0137         this.drawTile(graphics, 24, 7, 'stone', 46, 24);
0138         this.drawBlock(graphics, -24, 2, 17, 18);
0139         this.drawBlock(graphics, 24, 16, 17, 18);
0140         this.drawChevron(graphics, -4, 4, 6, 2);
0141         this.drawChevron(graphics, 5, 12, -6, -2);
0142     }
0143
0144     private drawTile(
0145         graphics: Graphics,
0146         x: number,
0147         y: number,
0148         kind: 'stone' | 'glass' | 'bridge',
0149         width: number,
0150         height: number,
0151     ): void {
0152         const halfWidth = width * 0.5;
0153         const halfHeight = height * 0.5;
0154         const depth = 9;
0155         const top: Point[] = [[x, y + halfHeight], [x + halfWidth, y], [x, y - halfHeight], [x - halfWidth, y]];
0156         const right: Point[] = [[x + halfWidth, y], [x, y - halfHeight], [x, y - halfHeight - depth], [x + halfWidth, y - depth]];
0157         const left: Point[] = [[x - halfWidth, y], [x, y - halfHeight], [x, y - halfHeight - depth], [x - halfWidth, y - depth]];
0158         const topColor = kind === 'glass' ? COLORS.glassTop : kind === 'bridge' ? COLORS.bridgeTop : COLORS.stoneTop;
0159         const sideColor = kind === 'glass' ? COLORS.glassSide : kind === 'bridge' ? COLORS.bridgeSide : COLORS.stoneSide;
0160         const edgeColor = kind === 'glass' ? COLORS.glassEdge : COLORS.stoneEdge;
0161         this.fillPolygon(graphics, left, sideColor);
0162         this.fillPolygon(graphics, right, this.darken(sideColor, 0.78));
0163         this.fillPolygon(graphics, top, topColor);
0164         this.strokePolygon(graphics, top, edgeColor, kind === 'glass' ? 2 : 1);
0165     }
0166
0167     private drawHole(graphics: Graphics, x: number, y: number, width: number, height: number): void {
0168         const halfWidth = width * 0.5;
0169         const halfHeight = height * 0.5;
0170         const rim: Point[] = [[x, y + halfHeight], [x + halfWidth, y], [x, y - halfHeight], [x - halfWidth, y]];
0171         this.fillPolygon(graphics, rim, COLORS.goatRim);
0172         const inset = 5;
0173         this.fillPolygon(graphics, [
0174             [x, y + halfHeight - inset],
0175             [x + halfWidth - inset * 1.7, y],
0176             [x, y - halfHeight + inset],
0177             [x - halfWidth + inset * 1.7, y],
0178         ], COLORS.goat);
0179     }
0180
0181     private drawBlock(graphics: Graphics, x: number, y: number, width: number, height: number): void {
0182         const halfWidth = width * 0.5;
0183         const topHeight = width * 0.28;
0184         const left: Point[] = [[x - halfWidth, y + height], [x, y + height - topHeight], [x, y - topHeight], [x - halfWidth, y]];
0185         const right: Point[] = [[x + halfWidth, y + height], [x, y + height - topHeight], [x, y - topHeight], [x + halfWidth, y]];
```

```
0186 |     const top: Point[] = [[x, y + height + topHeight], [x + halfWidth, y + height], [x, y + height - topHeight], [x - halfWidth, y +
height]];
0187 |     this.fillPolygon(graphics, left, COLORS.blockLeft);
0188 |     this.fillPolygon(graphics, right, COLORS.blockRight);
0189 |     this.fillPolygon(graphics, top, COLORS.blockTop);
0190 |     this.strokePolygon(graphics, [...left, ...right.slice(1), ...top.slice(1)], COLORS.blockEdge, 0.8);
0191 | }
0192 |
0193 | private drawChevron(graphics: Graphics, x: number, y: number, dx: number, dy: number): void {
0194 |     const perpendicularX = dy * 0.55;
0195 |     const perpendicularY = -dx * 0.55;
0196 |     graphics.strokeColor = COLORS.guide;
0197 |     graphics.lineWidth = 2;
0198 |     graphics.moveTo(x - dx + perpendicularX, y - dy + perpendicularY);
0199 |     graphics.lineTo(x, y);
0200 |     graphics.lineTo(x - dx - perpendicularX, y - dy - perpendicularY);
0201 |     graphics.stroke();
0202 | }
0203 |
0204 | private fillPolygon(graphics: Graphics, points: readonly Point[], color: Color): void {
0205 |     graphics.fillColor = color;
0206 |     graphics.moveTo(points[0][0], points[0][1]);
0207 |     for (let index = 1; index < points.length; index += 1) graphics.lineTo(points[index][0], points[index][1]);
0208 |     graphics.close();
0209 |     graphics.fill();
0210 | }
0211 |
0212 | private strokePolygon(graphics: Graphics, points: readonly Point[], color: Color, width: number): void {
0213 |     graphics.strokeColor = color;
0214 |     graphics.lineWidth = width;
0215 |     graphics.moveTo(points[0][0], points[0][1]);
0216 |     for (let index = 1; index < points.length; index += 1) graphics.lineTo(points[index][0], points[index][1]);
0217 |     graphics.close();
0218 |     graphics.stroke();
0219 | }
0220 |
0221 | private drawProjectedStoneTile(graphics: Graphics): void {
0222 |     const topY = 0.1;
0223 |     const bottomY = -0.1;
0224 |     const northSide: BoardPoint[] = [
0225 |         [-0.5, topY, -0.5], [0.5, topY, -0.5],
0226 |         [0.5, bottomY, -0.5], [-0.5, bottomY, -0.5],
0227 |     ];
0228 |     const westSide: BoardPoint[] = [
0229 |         [-0.5, topY, 0.5], [-0.5, topY, -0.5],
0230 |         [-0.5, bottomY, -0.5], [-0.5, bottomY, 0.5],
0231 |     ];
0232 |     const stoneTop: BoardPoint[] = [
0233 |         [-0.492, 0.1225, 0.492], [0.492, 0.1225, 0.492],
0234 |         [0.492, 0.1225, -0.492], [-0.492, 0.1225, -0.492],
0235 |     ];
0236 |
0237 |     this.fillPolygon(graphics, northSide.map((point) => this.projectBoardPoint(point)), this.darken(COLORS.stoneSide, 0.78));
0238 |     this.fillPolygon(graphics, westSide.map((point) => this.projectBoardPoint(point)), COLORS.stoneSide);
```

```
0239     const projectedTop = stoneTop.map((point) => this.projectBoardPoint(point));
0240     this.fillPolygon(graphics, projectedTop, COLORS.stoneTop);
0241     this.strokePolygon(graphics, projectedTop, COLORS.stoneEdge, 1);
0242 }
0243
0244 private projectedBoxTop(width: number, depth: number, y: number, yaw: number): Point[] {
0245     const halfWidth = width * 0.5;
0246     const halfDepth = depth * 0.5;
0247     return [
0248         this.rotateBoardPoint(-halfWidth, halfDepth, y, yaw),
0249         this.rotateBoardPoint(halfWidth, halfDepth, y, yaw),
0250         this.rotateBoardPoint(halfWidth, -halfDepth, y, yaw),
0251         this.rotateBoardPoint(-halfWidth, -halfDepth, y, yaw),
0252     ].map((point) => this.projectBoardPoint(point));
0253 }
0254
0255 private projectedTriangle(centerX: number, centerZ: number, radius: number, y: number, yaw: number): Point[] {
0256     const radians = yaw * Math.PI / 180;
0257     const cosine = Math.cos(radians);
0258     const sine = Math.sin(radians);
0259     const points: BoardPoint[] = [];
0260     for (let index = 0; index < 3; index += 1) {
0261         const theta = index / 3 * Math.PI * 2;
0262         const localX = Math.sin(theta) * radius;
0263         const localZ = Math.cos(theta) * radius;
0264         points.push([
0265             centerX + localX * cosine + localZ * sine,
0266             y,
0267             centerZ - localX * sine + localZ * cosine,
0268         ]);
0269     }
0270     return points.map((point) => this.projectBoardPoint(point));
0271 }
0272
0273 private rotateBoardPoint(x: number, z: number, y: number, yaw: number): BoardPoint {
0274     const radians = yaw * Math.PI / 180;
0275     const cosine = Math.cos(radians);
0276     const sine = Math.sin(radians);
0277     return [x * cosine + z * sine, y, -x * sine + z * cosine];
0278 }
0279
0280 private projectBoardPoint([x, y, z]: BoardPoint): Point {
0281     return [
0282         (x * BOARD_SCREEN_X[0] + y * BOARD_SCREEN_Y[0] + z * BOARD_SCREEN_Z[0]) * BOARD_PROJECTION_SCALE,
0283         (x * BOARD_SCREEN_X[1] + y * BOARD_SCREEN_Y[1] + z * BOARD_SCREEN_Z[1]) * BOARD_PROJECTION_SCALE,
0284     ];
0285 }
0286
0287 private darken(color: Color, factor: number): Color {
0288     return new Color(color.r * factor, color.g * factor, color.b * factor, color.a);
0289 }
0290 }
```